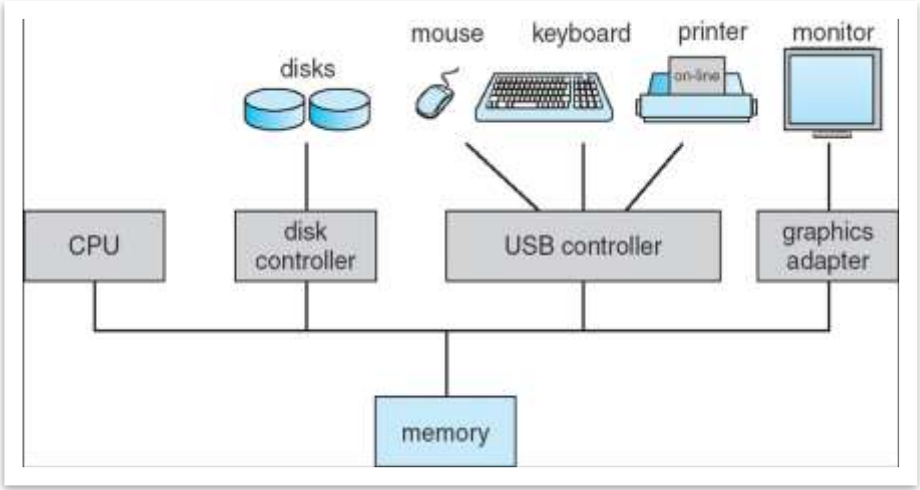
Introduction to OS

CSE-307

Lec Sameha Kamrul

MODERN COMPUTER SYSTEM

- It is a complex system
  - Many H/W components
  - Many programs running simultaneously



- Each running program
  - Executed in the CPU
  - Resides in the Memory
  - May interact with several devices

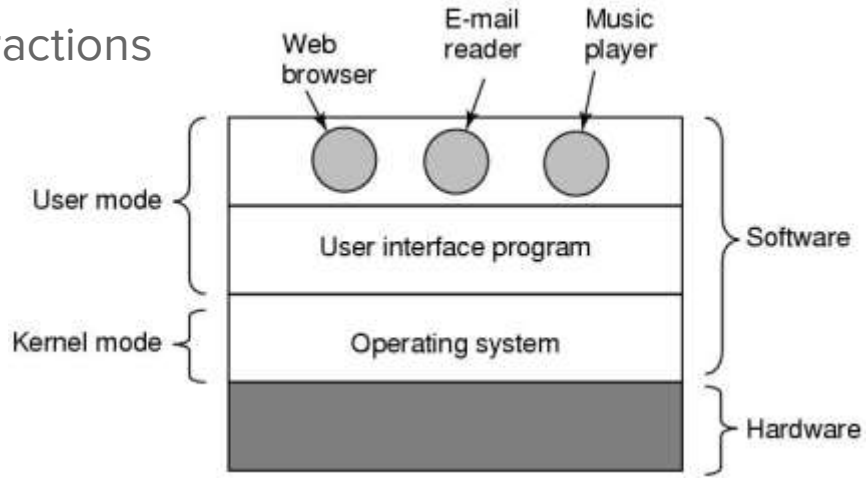
MODERN COMPUTER SYSTEM

- To ensure the correct operation someone needs to
  - Understand how all these components work
  - Manage them wisely
  - Allocate them efficiently

- So, computers are equipped with a **special software**
  - THE ***OPERATING SYSTEM***

OPERATING SYSTEM

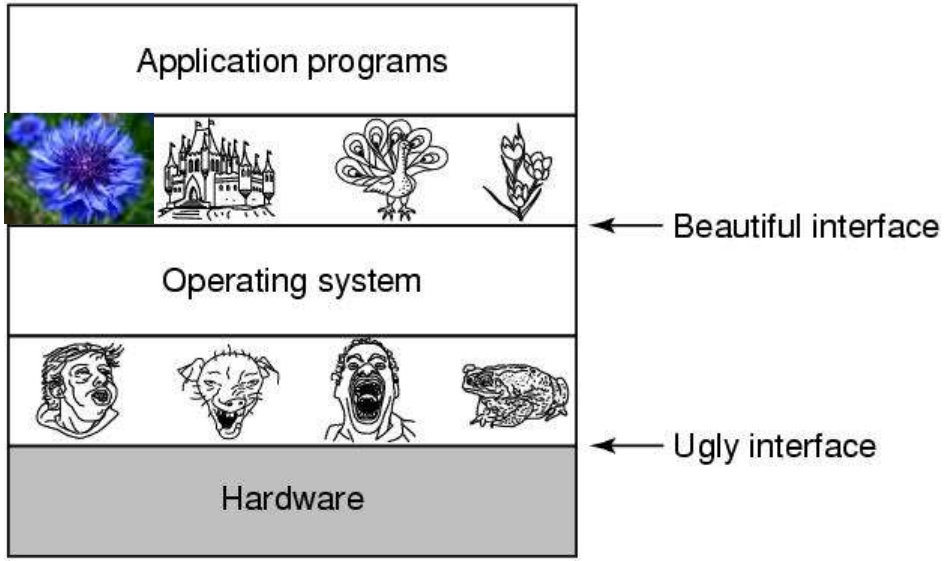
- A collection of exceptionally complex programs
- Runs on the bare hardware and provides the base for all the other software
- Makes the computer system **easy to use**
- Providing standard services and abstractions



ROLE OF OS

- An extended machine
  - Provide application programmers a clean abstract set of resources instead of the messy hardware ones
- A resource manager
  - Manage the hardware resources

AN EXTENDED MACHINE



Operating systems turn ugly hardware into beautiful abstractions

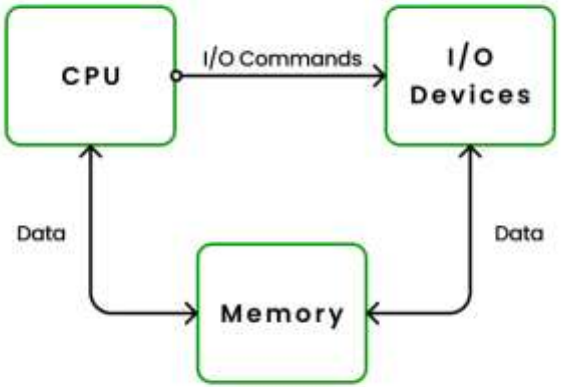
AN EXTENDED MACHINE

- Direct communication with hardware is very difficult
  - Need to understand all the complexities of simple operations
  - Need low level programming
- What programmer wants is a simple, high-level abstraction to deal with hardware
- The job of the OS
  - Create good abstractions
  - Implement and manage the abstract objects
- Example:
  - OS allow programmers to read write files without concern for how precisely they get stored in the hardware.

A RESOURCE MANAGER

- Allows multiple programs to run at the same time
- Provide an orderly and controlled allocation of
  - Processors
  - Memories
  - I/0 devices

  for the competing programs



- Example:
  - 3 programs running on the same computer
  - All tried to print their output simultaneously on the same printer
  - The OS will ensure that the printing is done orderly.

A RESOURCE MANAGER

- Resources can be shared in two different ways:
  - **Time** multiplexing:
    - In a single CPU system, several running programs take turns in running the CPU.
  - **Space** multiplexing:
    - main memory is divided up among several running programs.
- OS has to implement these multiplexing

OS Services

- Who are the real customers of OS?
  - End users
  - Application programmers who build different services for end users

SOME OS FUNCTIONS-

- Process Management
- Memory Management
- I/O Management
- Deadlock Management
- File system Management

OS Evolution

OS evolved along the way with the following concepts chronologically from top to bottom

- Uniprogramming
    - Serial Processing
    - Simple Batch System
- Multiprogramming

OS Evolution

Uniprogramming

Serial Processing

- The programmer interacts directly with the computer hardware, there is no OS, and only a single job was performed at a time.
- Limitations: setting up of required resources into the hardware, taking up a considerable amount of time.

OS Evolution

Uniprogramming

Simple Batch System

■ Monitor, a naive OS, prevented direct access of the user to the processor.

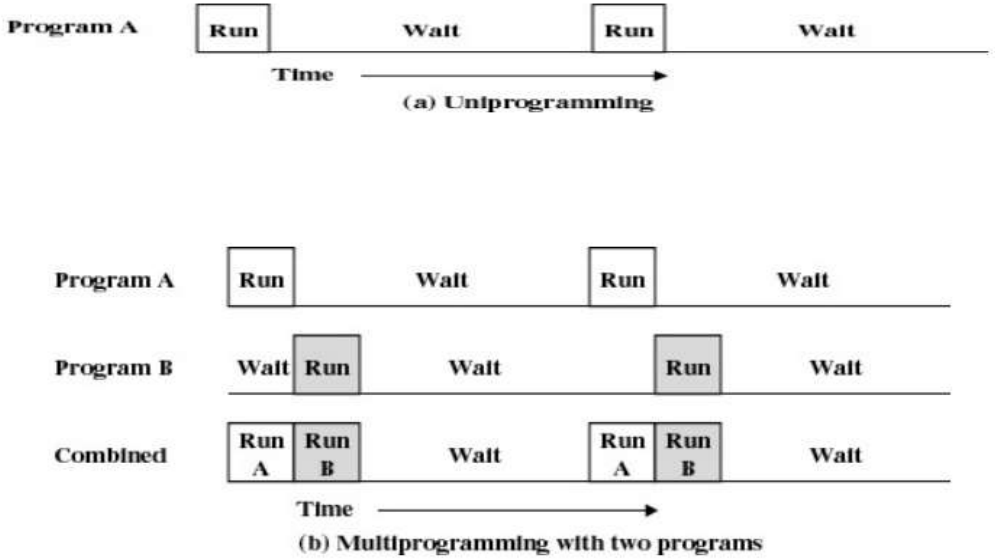
Entire batch of jobs are submitted to the monitor by an input device(Card Reader). Control is passed to this Monitor. This monitor causes job execution one by one and prints the job outputs on output device.

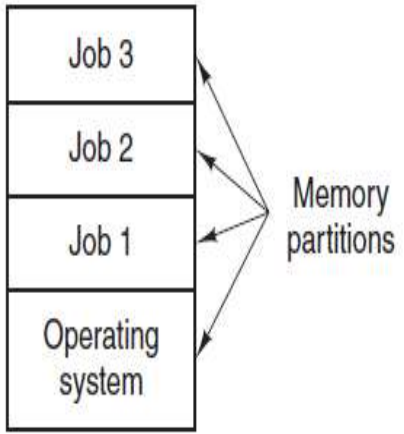
■ Limitations: A user needs to write extra code to tell the monitor to how to allow the execution of the job.

OS Evolution

Multiprogramming

- In uniprogramming, next program start executing after the current program completed or stopped execution. But in multiprogramming another program can run while the current is doing I/O.





A multiprogramming system with three jobs in memory.

Booting the computer

Power On: As soon as the power is turned on, CPU starts executing instructions from the firmware (or system BIOS) in non-volatile ROM/Flash memory. Firmware is a special program in the flash memory.

Firmware's Job: Firmware initializes basic hardware and then finds a bootable device (e.g., your hard drive).

Bootloader Loading: The firmware then loads the bootloader program from the boot sector of the bootable device into RAM/main memory.

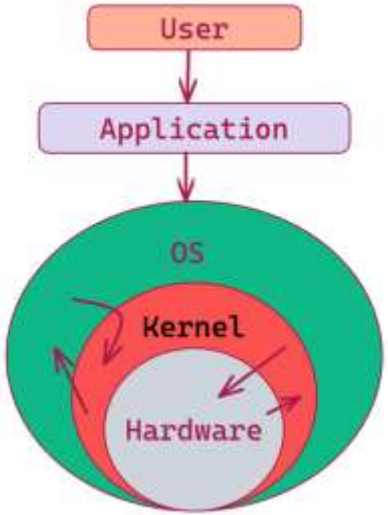
Booting the computer

Kernel Loading: The bootloader (now running in RAM) reads its configuration, finds the compiled kernel image on the hard drive and loads them into other designated locations in RAM.

Kernel Execution: The bootloader then transfers control to the loaded kernel. The kernel code (now in RAM) begins execution. The kernel then starts up a login program or GUI.

KERNEL

- The most fundamental part of an OS
- Running at all times on the computer
- Often used as another name for OS (however not factually correct)
- Typical components
  - interrupt handlers
  - schedulers
  - memory management unit
  - system services



- Parts of the OS, but not the kernel
  - Shell
  - System libraries
  - In-built application softwares (web browsers, games, etc.)

DUAL-MODE OPERATION

- CPU executes in 2 Modes
  - Kernel mode (Privileged mode)
    - CPU can execute all machine instructions
    - CPU can use every hardware feature
  - User mode
    - permits only a subset of the instructions and a subset of the hardware features to CPU
    - this mode is safer and cannot cause an entire system crash

DUAL-MODE OPERATION

- OS runs in kernel mode, user programs in user mode
  - This allows the OS to protect itself and other system components.

- Mode bit provided by **hardware**
  - Provides ability to distinguish when system is running user code or kernel code.

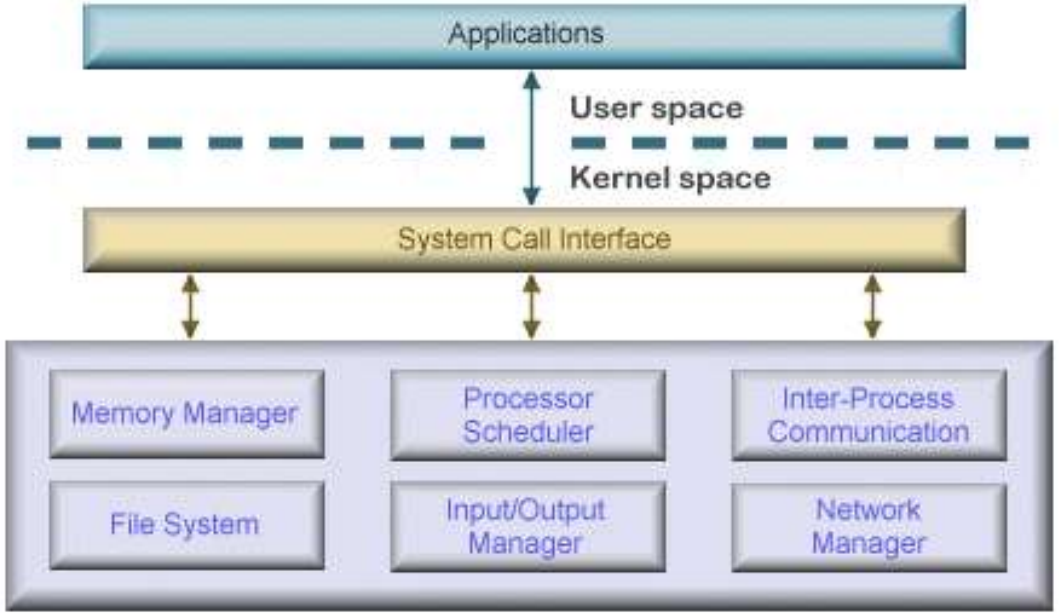
- CPU Execution switching between user and kernel mode is known as **Mode-switching**

DUAL-MODE OPERATION

- When does the CPU starts executing in kernel mode?
  - During a System Boot
    - Starting a computer
  - Hardware Interrupt
    - generated by hardware devices to signal that they need some attention from the OS
  - Trap
    - It is a software-generated interrupt caused by
      - an error (such as division by 0 or invalid memory access) or
      - a specific request from a user program for an operating-system service

DUAL-MODE OPERATION

- To obtain services from the operating system, a user program must make a ***system call***, which ***traps*** into the kernel and invokes the operating system.



SYSTEM CALLS

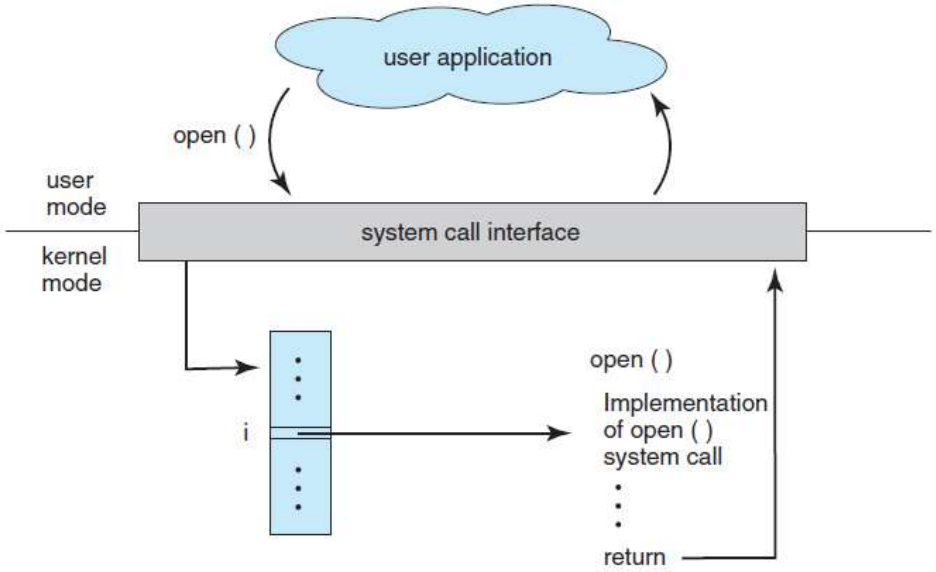
- Programming interfaces to the services provided by the OS
- Typically written in a high-level language (C or C++)
- They are mostly accessed by programs via Application Program Interface (API) rather than direct system call use
- Two most common APIs:

    1. Win32 API for Windows
    2. POSIX API for UNIX, Linux, Mac OS X

- For eg, the printf( ) C standard library procedure uses the write( ) procedure of the POSIX API, which then uses the write system call *(system calls and their corresponding API procedure may have the same name)* for writing output into a console.

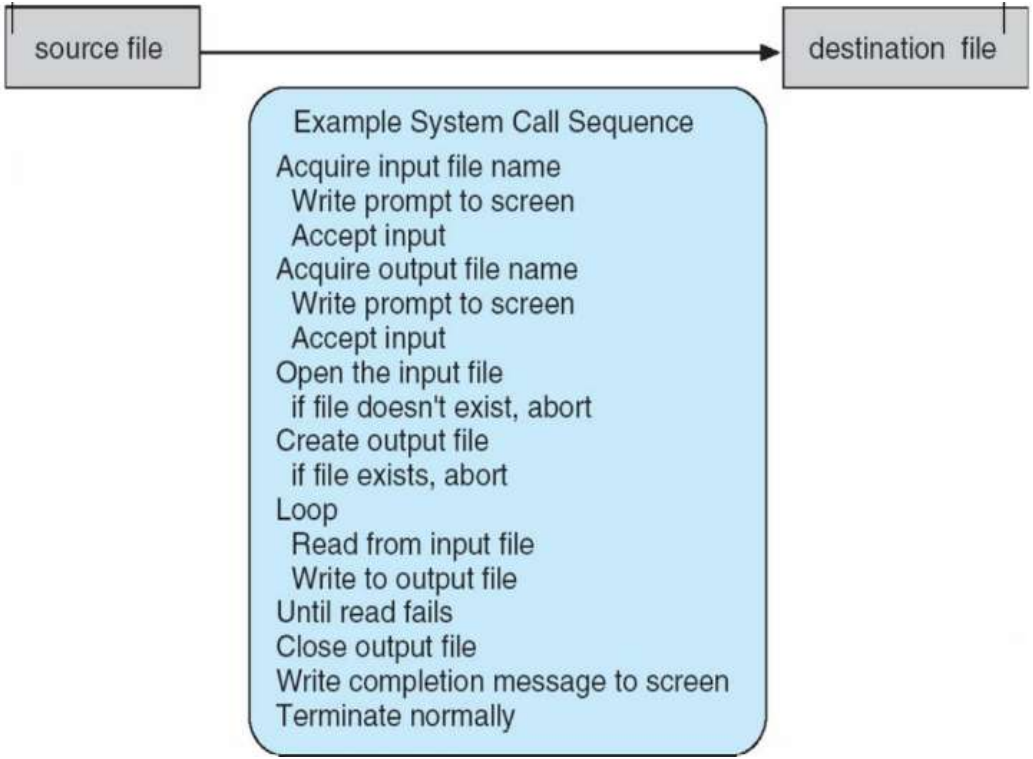
SYSTEM CALLS

- Typically a number is associated with each system call, and the OS maintains a table indexed according to these numbers.



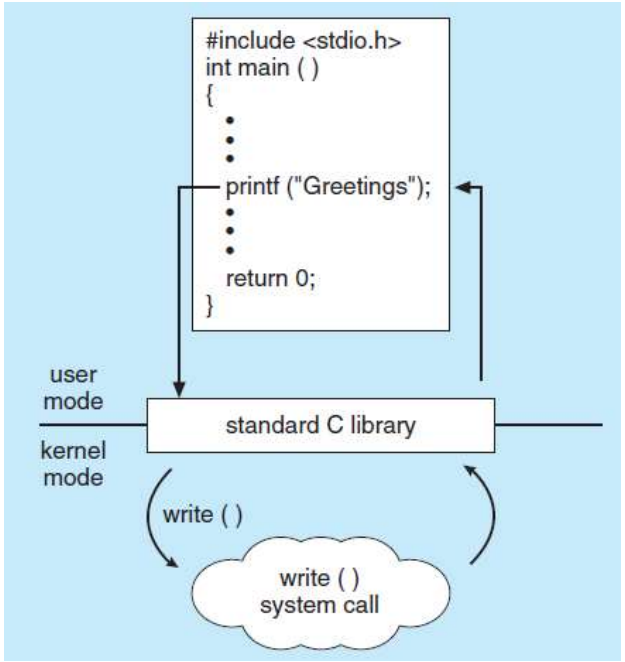
SYSTEM CALLS

Example of a **sequence of system calls** for copying the contents of a file into a new file



SYSTEM CALLS

- If a process is running a user program in user mode and needs a system service, then it has to execute an assembly trap instruction to transfer control to the operating system.
- The operating system then figures out what the calling process wants by inspecting the parameters, then OS carries out the system call.
- After that, OS returns control to the instruction following the system call. Hence, system call is a special kind of procedure call except that it involves mode-switching.



SYSTEM CALLS

Three general methods for passing system call parameters:

Method#1: Pass parameters in registers.

Method#2: Store the parameters in a table in memory, and the table address is passed as a parameter in a register.

Method#3: Push (store) the parameters into the stack by the program, and pop off the stack by operating system.

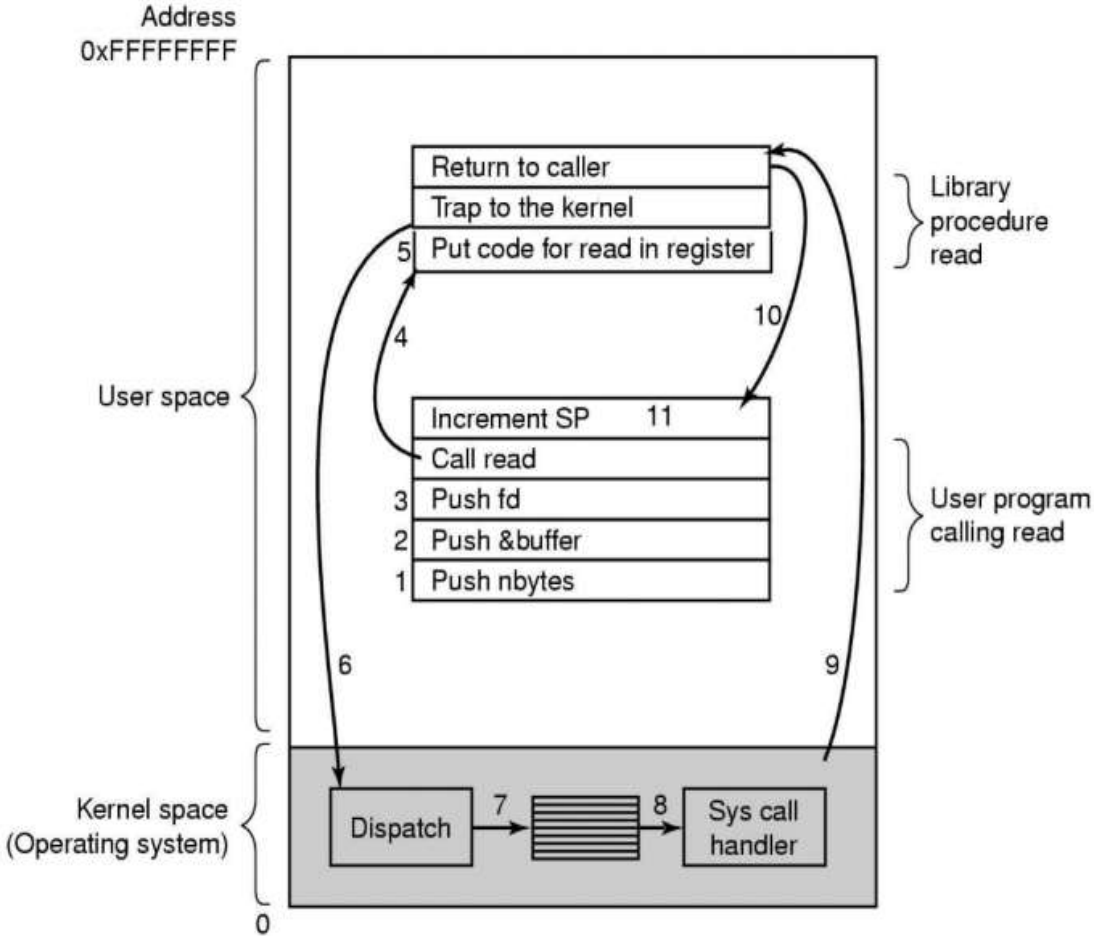


Figure 1-17. The 11 steps in making the system call read(fd, buffer, nbytes).

SYSTEM CALLS

1. C and C++ compilers pass parameters between a running program and the operating system using parameter passing method#3. (Steps: 1-3)
2. Actual call to library procedure happens. (Step: 4)
3. The library procedure puts the system call number in a register, where the OS will expect it to be. (step 5)
4. A TRAP instruction is performed to switch from user mode to kernel mode. (step 6)

SYSTEM CALLS

5. The kernel examines the system call number and then dispatches to the correct system call handler, via a table of pointers to system-call handlers indexed by the system-call number. (step 7)
6. System call handler runs. (step 8)
7. Control returned to the user-space library procedure at the instruction following the TRAP instruction. (step 9)
8. This library procedure then returns to the user program in the usual way procedure calls return. (Step 10)
9. The user program has to clean up the stack by incrementing Stack pointer value, as it does after any procedure call. (Step 11)

DIFFERENCE BETWEEN PROCEDURE CALL AND SYSTEM CALL

- Procedure call is done using the CALL instruction which jumps to an arbitrary address where the procedure code resides. The entire procedure call, execution, and its return takes place in the User Space.
- However, system call is done using the TRAP instruction which always jumps to a single fixed location in the kernel code. System call also involves switching from user space to kernel space.

SYSTEM CALLS

System calls can be grouped roughly into **six** major categories:

Process control,

file manipulation,

device manipulation,

information maintenance,

communications, and

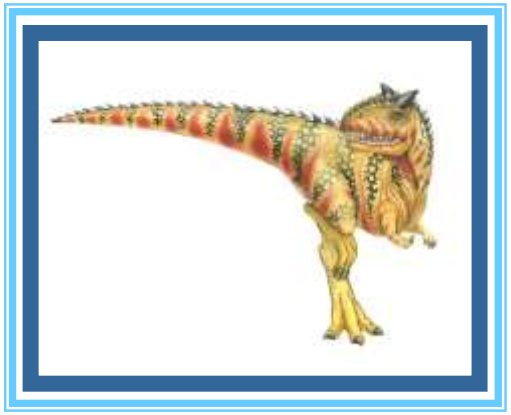
protection.

EXAMPLES OF WINDOWS AND UNIX SYSTEM CALLS

	Windows	Unix
Process Control	CreateProcess() ExitProcess() WaitForSingleObject()	fork() exit() wait()
File Manipulation	CreateFile() ReadFile() WriteFile() CloseHandle()	open() read() write() close()
Device Manipulation	SetConsoleMode() ReadConsole() WriteConsole()	ioctl() read() write()
Information Maintenance	GetCurrentProcessID() SetTimer() Sleep()	getpid() alarm() sleep()
Communication	CreatePipe() CreateFileMapping() MapViewOfFile()	pipe() shm_open() mmap()
Protection	SetFileSecurity() InitializeSecurityDescriptor() SetSecurityDescriptorGroup()	chmod() umask() chown()

Thank You

Chapter 3: Processes

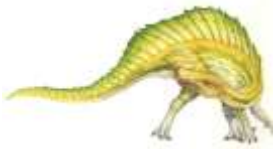


Silberschatz, Galvin and Gagne ©2013



Chapter 3: Processes

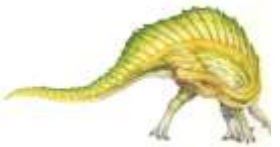
- Process Concept
- Process Scheduling
- Operations on Processes
- Interprocess Communication
- Examples of IPC Systems
- Communication in Client-Server Systems





Objectives

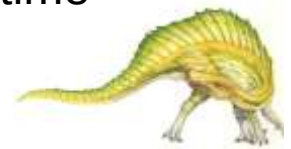
- To introduce the notion of a process -- a program in execution, which forms the basis of all computation
- To describe the various features of processes, including scheduling, creation and termination, and communication
- To explore interprocess communication using shared memory and message passing
- To describe communication in client-server systems





Process Concept

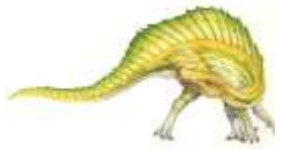
- An operating system executes a variety of programs:
  - Batch system – **jobs**
  - Time-shared systems – **user programs** or **tasks**
- Textbook uses the terms ***job*** and ***process*** almost interchangeably
- **Process** – a program in execution; process execution must progress in sequential fashion
- Multiple parts
  - The program code, also called **text section**
  - Current activity including **program counter**, processor registers
  - **Stack** containing temporary data
    - Function parameters, return addresses, local variables
  - **Data section** containing global variables
  - **Heap** containing memory dynamically allocated during run time





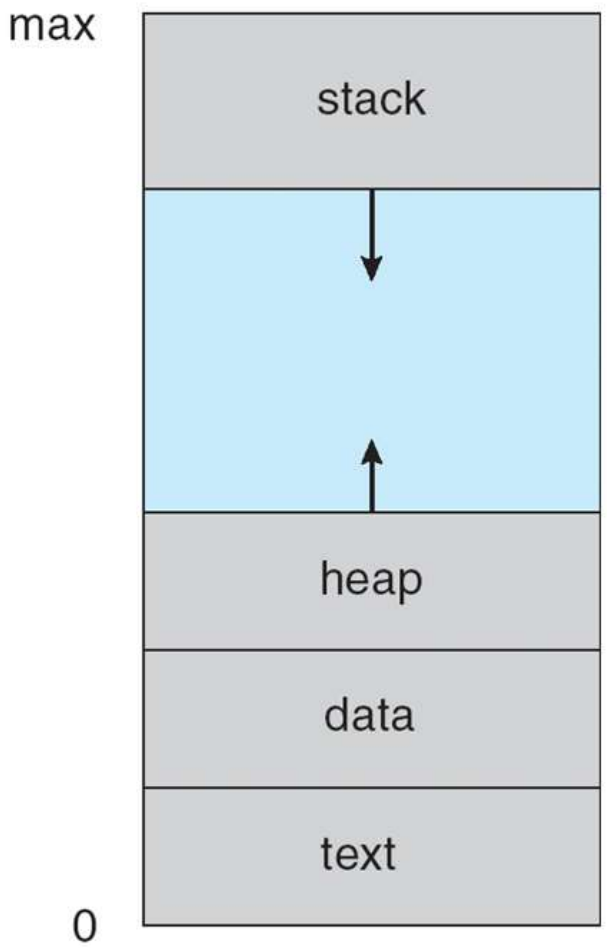
Process Concept (Cont.)

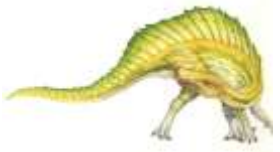
- Program is ***passive*** entity stored on disk (**executable file**), process is ***active***
  - Program becomes process when executable file loaded into memory
- Execution of program started via GUI mouse clicks, command line entry of its name, etc
- One program can be several processes
  - Consider multiple users executing the same program





Process in Memory







Process State

- As a process executes, it changes **state**
  - **new**: The process is being created
  - **running**: Instructions are being executed
  - **waiting**: The process is waiting for some event to occur
  - **ready**: The process is waiting to be assigned to a processor
  - **terminated**: The process has finished execution

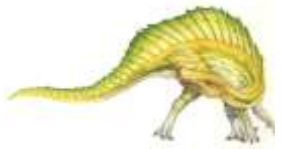
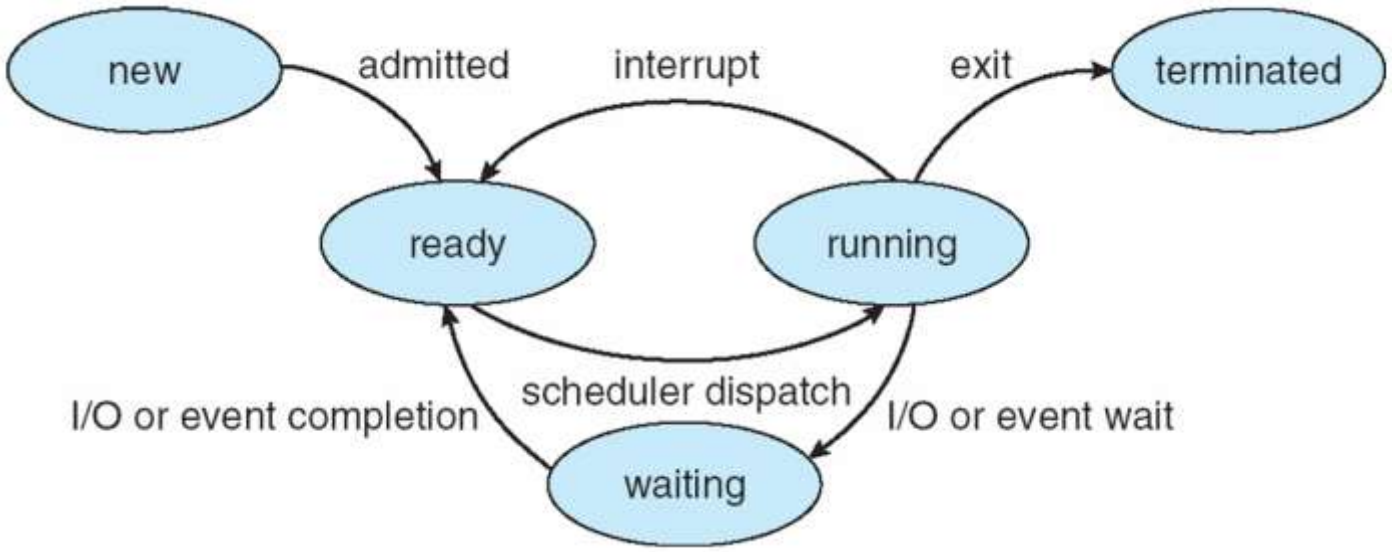
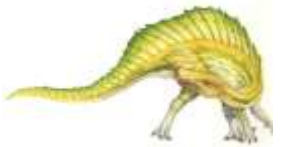




Diagram of Process State







Process Control Block (PCB)

Information associated with each process

(also called **task control block**)

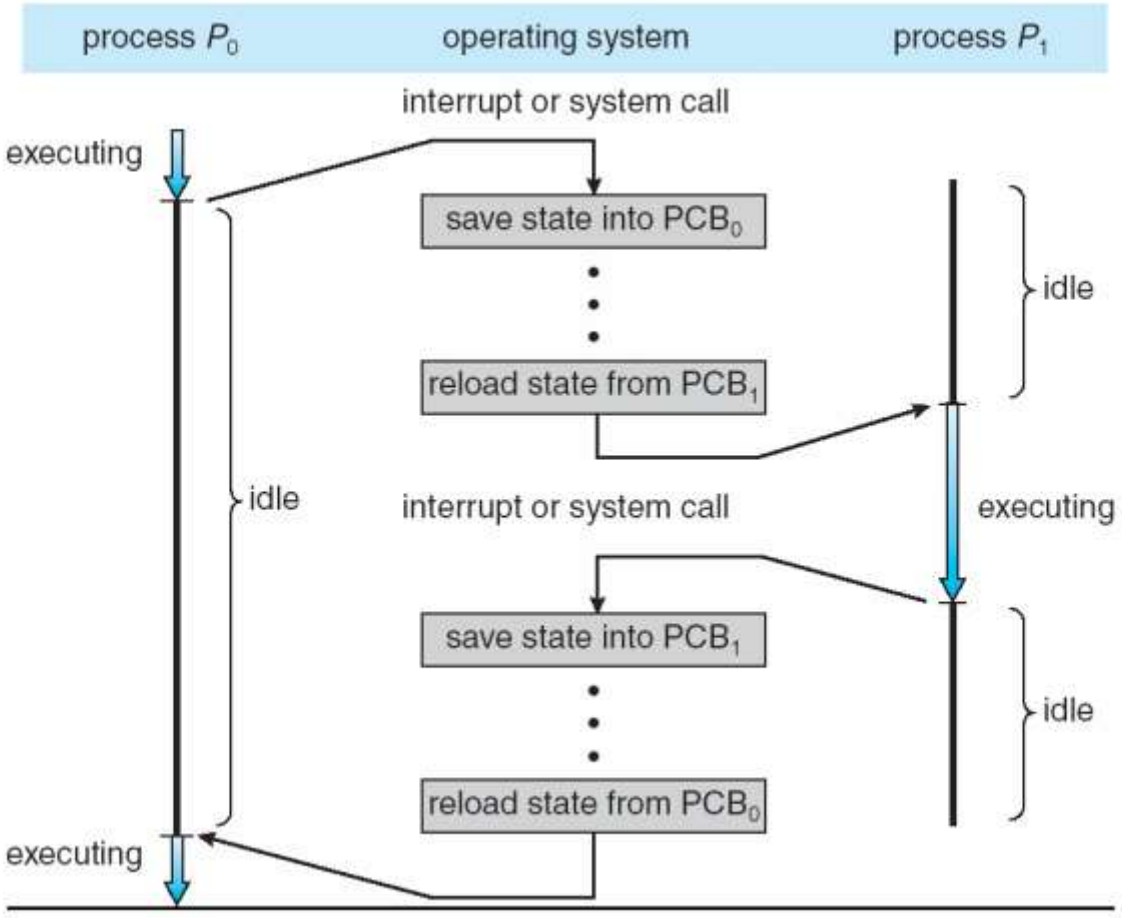
- Process state – running, waiting, etc
- Program counter – location of instruction to next execute
- CPU registers – contents of all process-centric registers
- CPU scheduling information- priorities, scheduling queue pointers
- Memory-management information – memory allocated to the process
- Accounting information – CPU used, clock time elapsed since start, time limits
- I/O status information – I/O devices allocated to process, list of open files

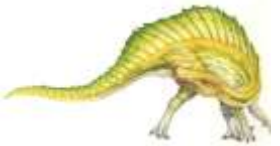






CPU Switch From Process to Process

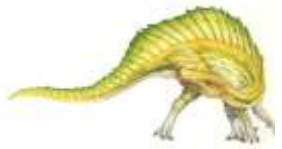






Threads

- So far, process has a single thread of execution
- Consider having multiple program counters per process
  - Multiple locations can execute at once
    - Multiple threads of control -> **threads**
- Must then have storage for thread details, multiple program counters in PCB
- See next chapter

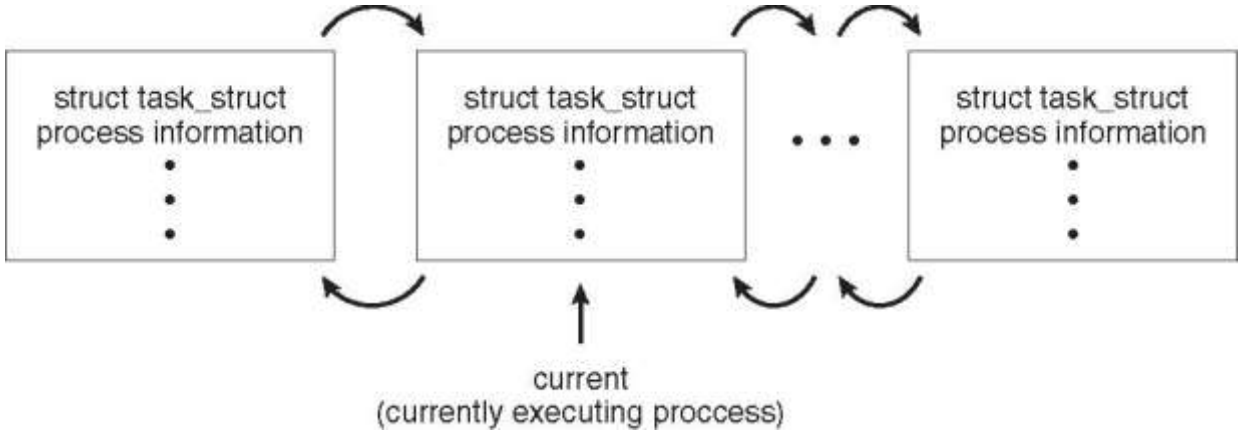


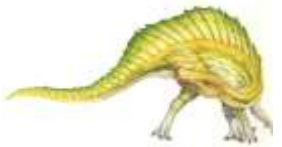


Process Representation in Linux

Represented by the C structure `task_struct`

```
pid t_pid; /* process identifier */
long state; /* state of the process */
unsigned int time_slice /* scheduling information */
struct task_struct *parent; /* this process's parent */
struct list_head children; /* this process's children */
struct files_struct *files; /* list of open files */
struct mm_struct *mm; /* address space of this process */
```

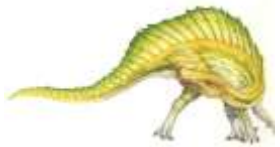






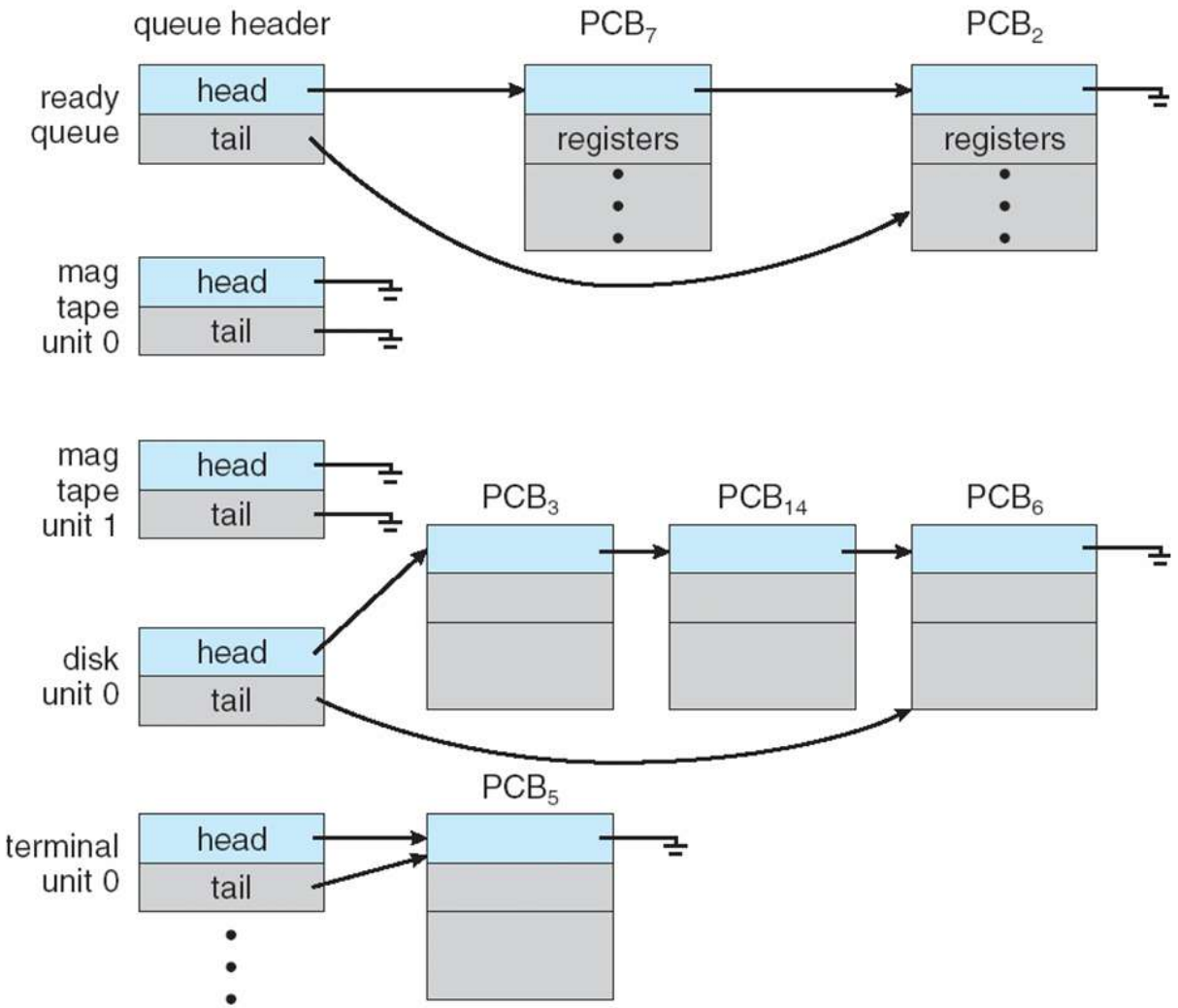
Process Scheduling

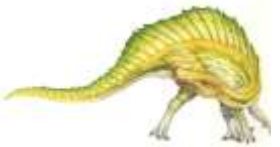
- Maximize CPU use, quickly switch processes onto CPU for time sharing
- **Process scheduler** selects among available processes for next execution on CPU
- Maintains **scheduling queues** of processes
  - **Job queue** – set of all processes in the system
  - **Ready queue** – set of all processes residing in main memory, ready and waiting to execute
  - **Device queues** – set of processes waiting for an I/O device
  - Processes migrate among the various queues





Ready Queue And Various I/O Device Queues

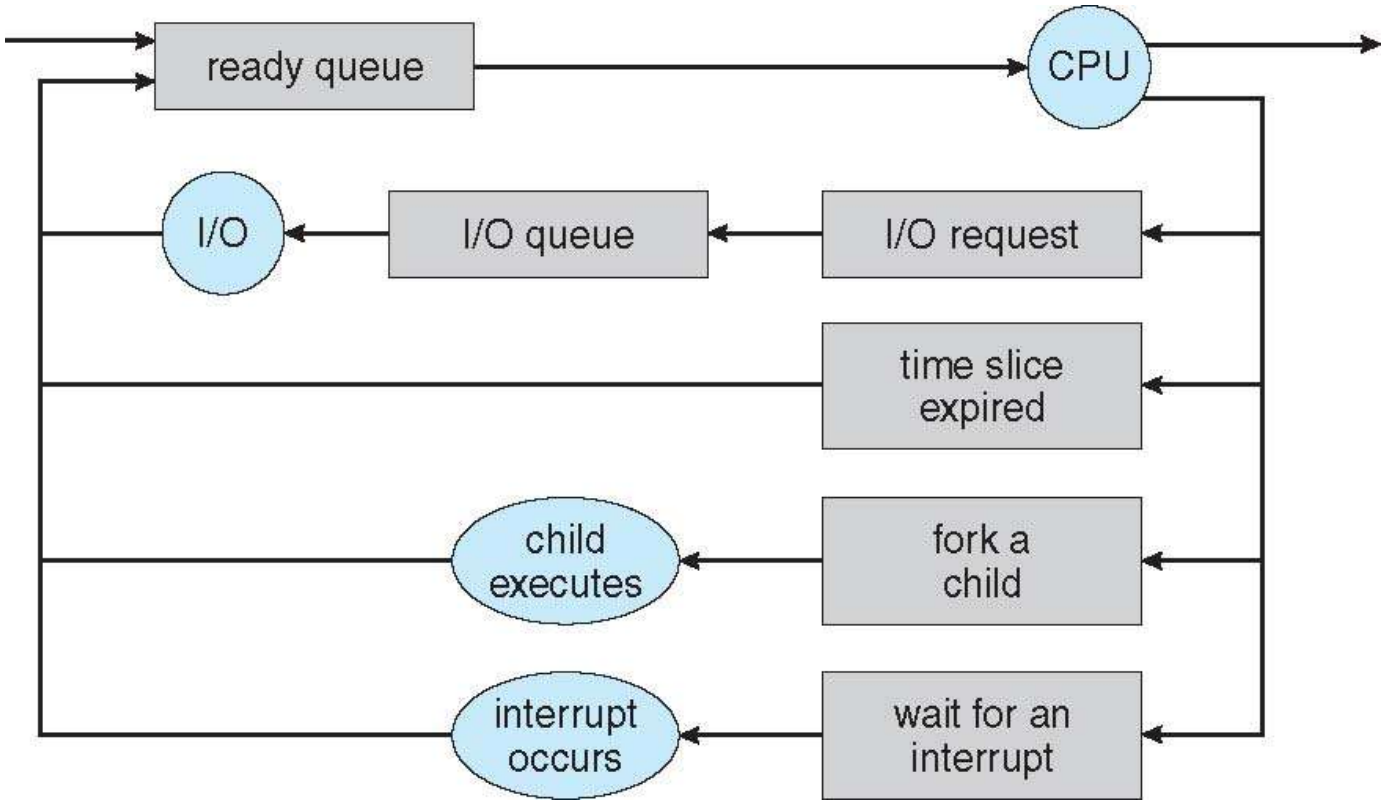


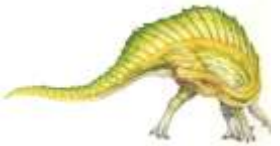




Representation of Process Scheduling

- **Queueing diagram** represents queues, resources, flows

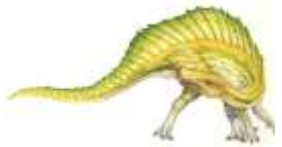






Schedulers

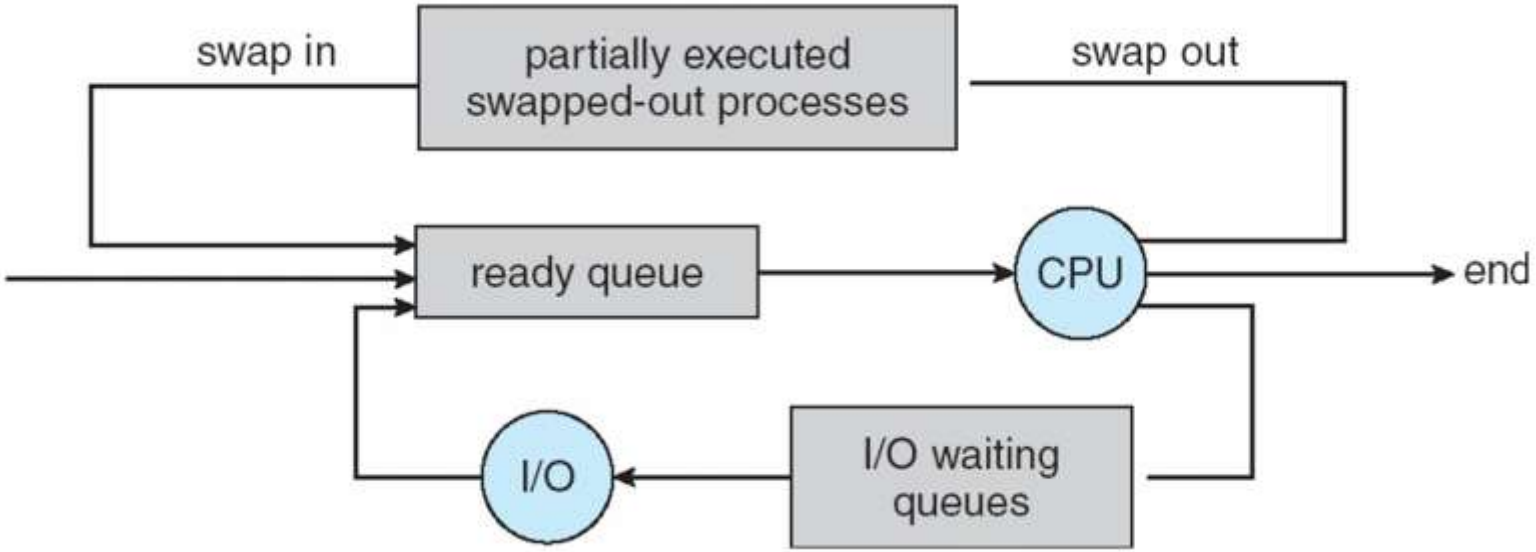
- **Short-term scheduler** (or **CPU scheduler**) – selects which process should be executed next and allocates CPU
  - Sometimes the only scheduler in a system
  - Short-term scheduler is invoked frequently (milliseconds) ⇒ (must be fast)
- **Long-term scheduler** (or **job scheduler**) – selects which processes should be brought into the ready queue
  - Long-term scheduler is invoked infrequently (seconds, minutes) ⇒ (may be slow)
  - The long-term scheduler controls the **degree of multiprogramming**
- Processes can be described as either:
  - **I/O-bound process** – spends more time doing I/O than computations, many short CPU bursts
  - **CPU-bound process** – spends more time doing computations; few very long CPU bursts
- Long-term scheduler strives for good ***process mix***

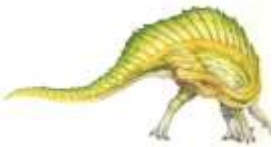




Addition of Medium Term Scheduling

- **Medium-term scheduler** can be added if degree of multiple programming needs to decrease
  - Remove process from memory, store on disk, bring back in from disk to continue execution: **swapping**

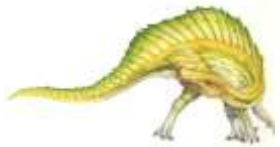






Multitasking in Mobile Systems

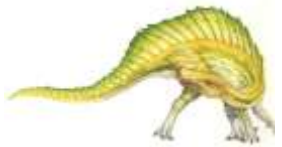
- Some mobile systems (e.g., early version of iOS) allow only one process to run, others suspended
- Due to screen real estate, user interface limits iOS provides for a
  - Single **foreground** process- controlled via user interface
  - Multiple **background** processes– in memory, running, but not on the display, and with limits
  - Limits include single, short task, receiving notification of events, specific long-running tasks like audio playback
- Android runs foreground and background, with fewer limits
  - Background process uses a **service** to perform tasks
  - Service can keep running even if background process is suspended
  - Service has no user interface, small memory use





Context Switch

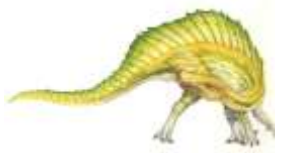
- When CPU switches to another process, the system must **save the state** of the old process and load the **saved state** for the new process via a **context switch**
- **Context** of a process represented in the PCB
- Context-switch time is overhead; the system does no useful work while switching
  - The more complex the OS and the PCB ➔ the longer the context switch
- Time dependent on hardware support
  - Some hardware provides multiple sets of registers per CPU ➔ multiple contexts loaded at once





Operations on Processes

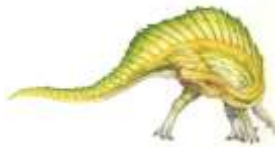
- System must provide mechanisms for:
  - process creation,
  - process termination,
  - and so on as detailed next





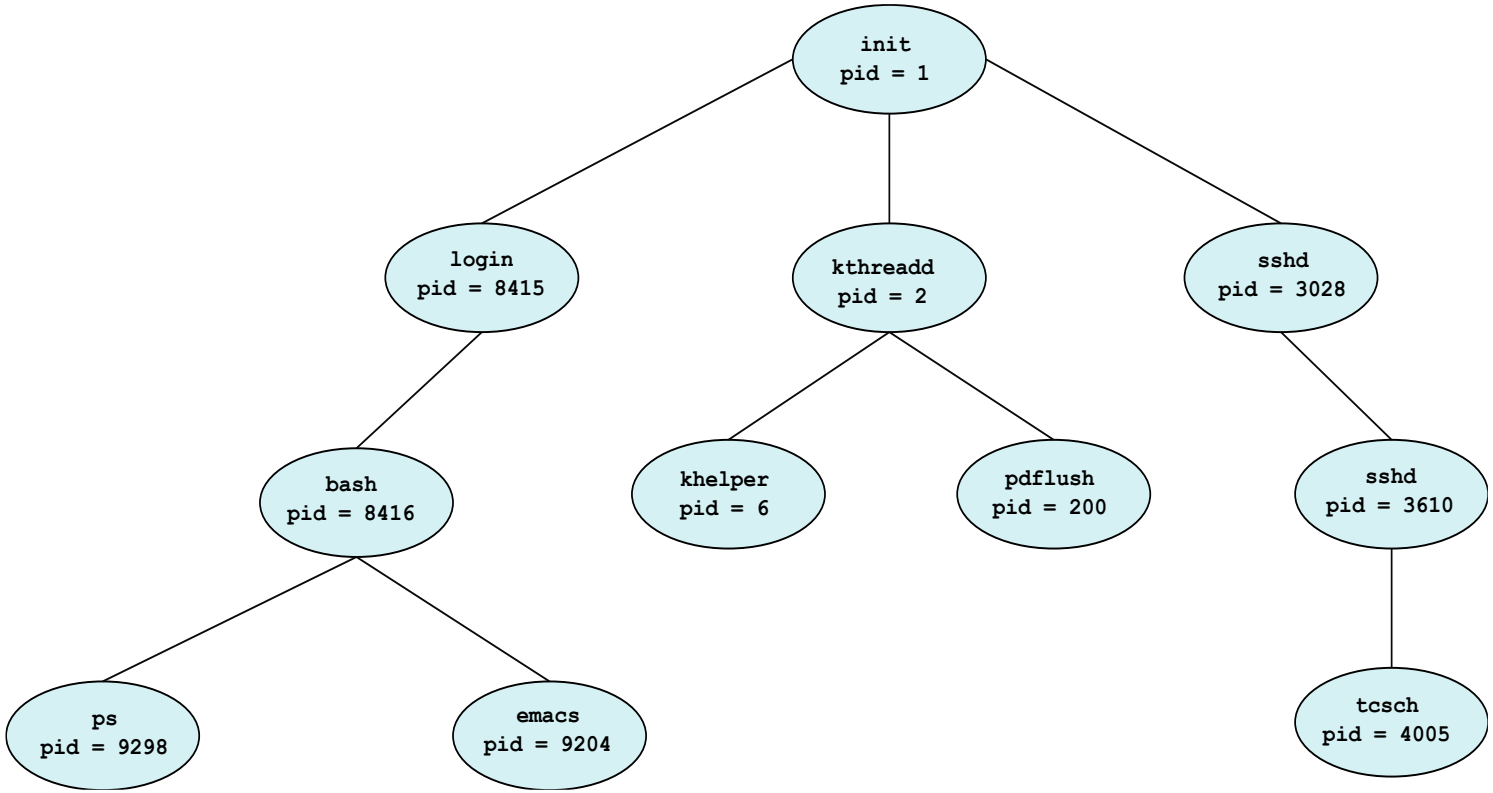
Process Creation

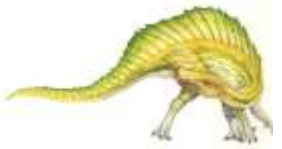
- **Parent** process create **children** processes, which, in turn create other processes, forming a **tree** of processes
- Generally, process identified and managed via a **process identifier** (**pid**)
- Resource sharing options
  - Parent and children share all resources
  - Children share subset of parent's resources
  - Parent and child share no resources
- Execution options
  - Parent and children execute concurrently
  - Parent waits until children terminate





A Tree of Processes in Linux

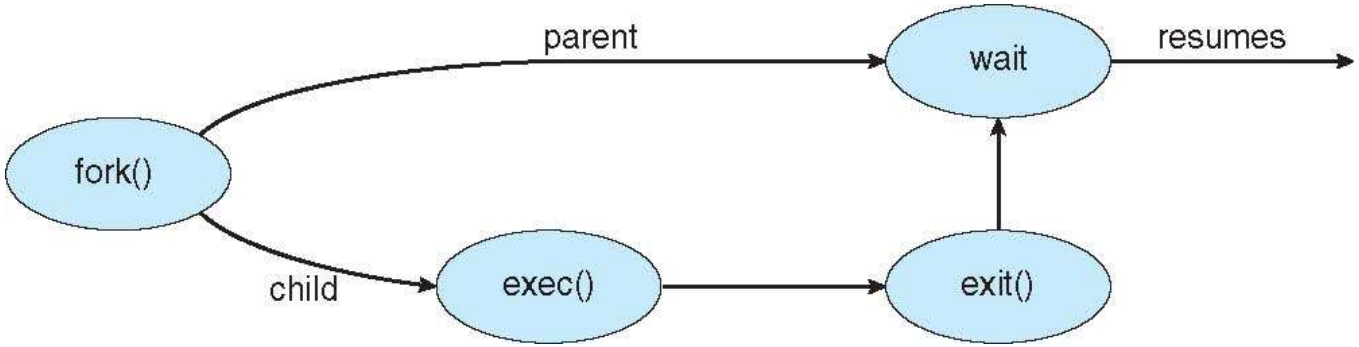


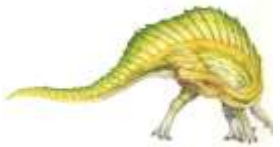




Process Creation (Cont.)

- Address space
  - Child duplicate of parent
  - Child has a program loaded into it
- UNIX examples
  - **`fork()`** system call creates new process
  - **`exec()`** system call used after a **`fork()`** to replace the process' memory space with a new program







C Program Forking Separate Process

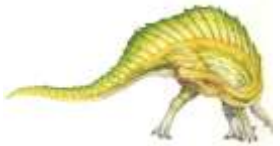
```
#include <sys/types.h>
#include <stdio.h>
#include <unistd.h>

int main()
{
pid_t pid;

    /* fork a child process */
    pid = fork();

    if (pid < 0) { /* error occurred */
       fprintf(stderr, "Fork Failed");
       return 1;
    }
    else if (pid == 0) { /* child process */
       execlp("/bin/ls","ls",NULL);
    }
    else { /* parent process */
       /* parent will wait for the child to complete */
       wait(NULL);
       printf("Child Complete");
    }

    return 0;
}
```





Creating a Separate Process via Windows API

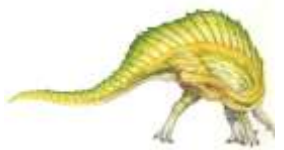
```
#include <stdio.h>
#include <windows.h>

int main(VOID)
{
STARTUPINFO si;
PROCESS_INFORMATION pi;

    /* allocate memory */
    ZeroMemory(&si, sizeof(si));
    si.cb = sizeof(si);
    ZeroMemory(&pi, sizeof(pi));

    /* create child process */
    if (!CreateProcess(NULL, /* use command line */
     "C:\\WINDOWS\\system32\\mspaint.exe", /* command */
     NULL, /* don't inherit process handle */
     NULL, /* don't inherit thread handle */
     FALSE, /* disable handle inheritance */
     0, /* no creation flags */
     NULL, /* use parent's environment block */
     NULL, /* use parent's existing directory */
     &si,
     &pi))
    {
       fprintf(stderr, "Create Process Failed");
       return -1;
    }
    /* parent will wait for the child to complete */
    WaitForSingleObject(pi.hProcess, INFINITE);
    printf("Child Complete");

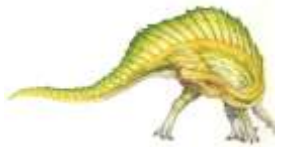
    /* close handles */
    CloseHandle(pi.hProcess);
    CloseHandle(pi.hThread);
}
```





Process Termination

- Process executes last statement and then asks the operating system to delete it using the **`exit()`** system call.
  - Returns status data from child to parent (via **`wait()`**)
  - Process' resources are deallocated by operating system
- Parent may terminate the execution of children processes using the **`abort()`** system call. Some reasons for doing so:
  - Child has exceeded allocated resources
  - Task assigned to child is no longer required
  - The parent is exiting and the operating systems does not allow a child to continue if its parent terminates



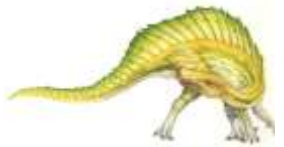


Process Termination

- Some operating systems do not allow child to exists if its parent has terminated. If a process terminates, then all its children must also be terminated.
  - **cascading termination.** All children, grandchildren, etc. are terminated.
  - The termination is initiated by the operating system.
- The parent process may wait for termination of a child process by using the `wait()` system call. The call returns status information and the pid of the terminated process

```
pid = wait(&status);
```

- If no parent waiting (did not invoke `wait()`) process is a **zombie**
- If parent terminated without invoking `wait`, process is an **orphan**

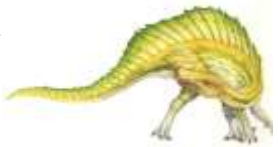




Multiprocess Architecture – Chrome Browser

- Many web browsers ran as single process (some still do)
  - If one web site causes trouble, entire browser can hang or crash
- Google Chrome Browser is multiprocess with 3 different types of processes:
  - **Browser** process manages user interface, disk and network I/O
  - **Renderer** process renders web pages, deals with HTML, Javascript. A new renderer created for each website opened
    - Runs in **sandbox** restricting disk and network I/O, minimizing effect of security exploits
  - **Plug-in** process for each type of plug-in

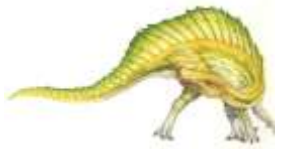






Interprocess Communication

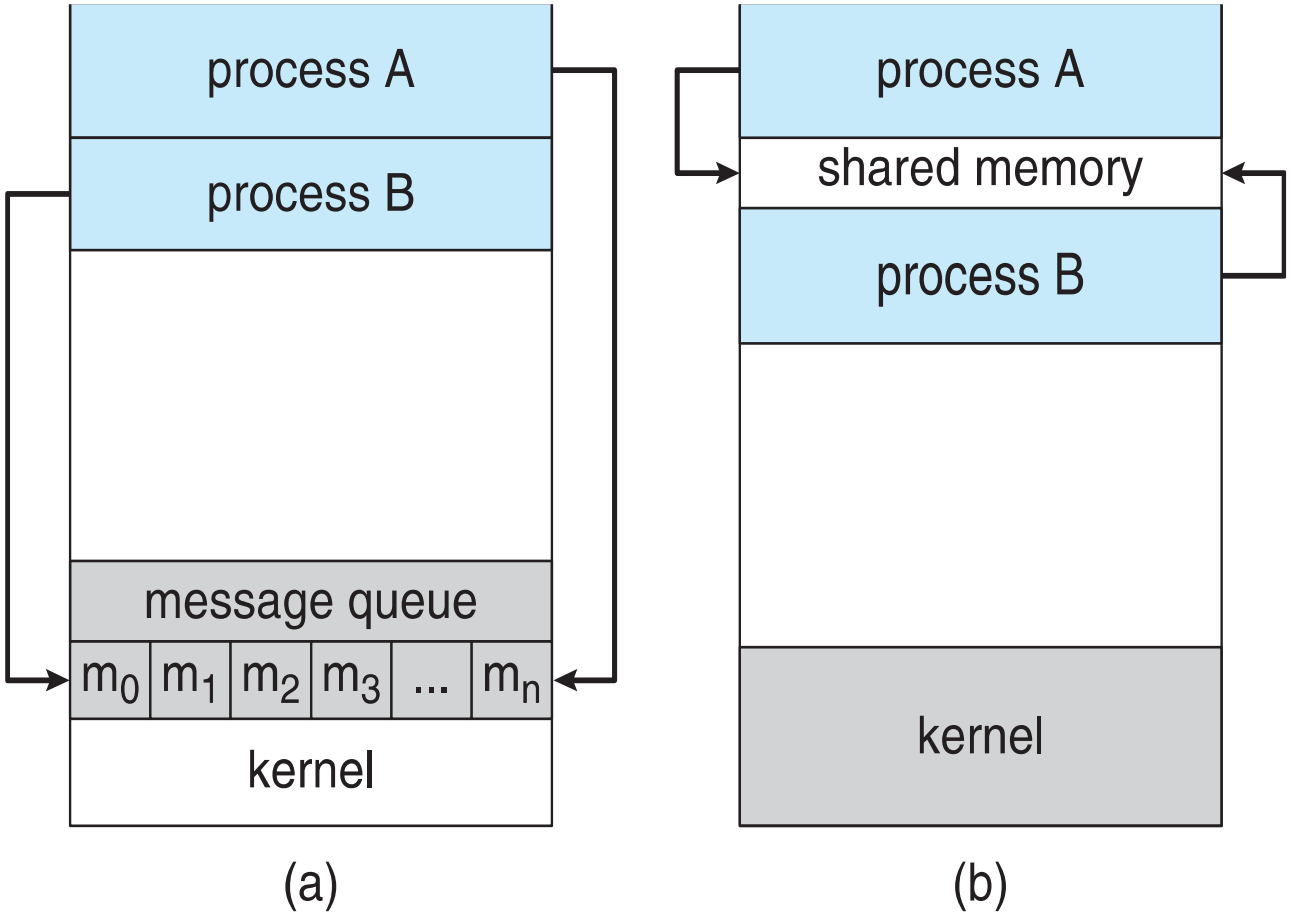
- Processes within a system may be ***independent*** or ***cooperating***
- Cooperating process can affect or be affected by other processes, including sharing data
- Reasons for cooperating processes:
  - Information sharing
  - Computation speedup
  - Modularity
  - Convenience
- Cooperating processes need **interprocess communication** (**IPC**)
- Two models of IPC
  - **Shared memory**
  - **Message passing**

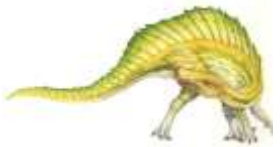




Communications Models

(a) Message passing. (b) shared memory.

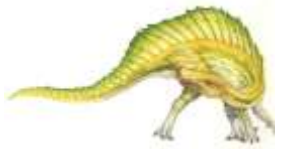






Cooperating Processes

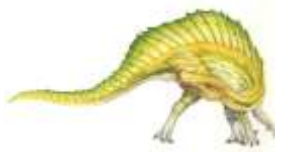
- ***Independent*** process cannot affect or be affected by the execution of another process
- ***Cooperating*** process can affect or be affected by the execution of another process
- Advantages of process cooperation
  - Information sharing
  - Computation speed-up
  - Modularity
  - Convenience





Producer-Consumer Problem

- Paradigm for cooperating processes, *producer* process produces information that is consumed by a *consumer* process
  - **unbounded-buffer** places no practical limit on the size of the buffer
  - **bounded-buffer** assumes that there is a fixed buffer size





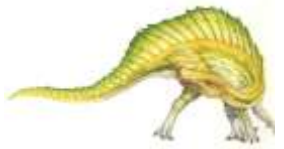
Bounded-Buffer – Shared-Memory Solution

- Shared data

```
#define BUFFER_SIZE 10
typedef struct {
  . . .
} item;

item buffer[BUFFER_SIZE];
int in = 0;
int out = 0;
```

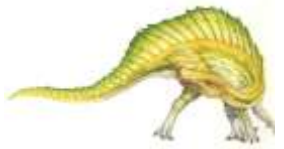
- Solution is correct, but can only use BUFFER_SIZE-1 elements





Bounded-Buffer – Producer

```
item next_produced;
while (true) {
        /* produce an item in next produced */
        while (((in + 1) % BUFFER_SIZE) == out)
                ; /* do nothing */
        buffer[in] = next_produced;
        in = (in + 1) % BUFFER_SIZE;
}
```





Bounded Buffer – Consumer

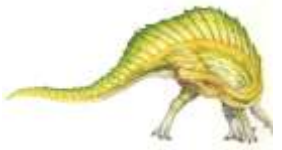
```
item next_consumed;

while (true) {
        while (in == out)
                ; /* do nothing */
        next_consumed = buffer[out];

        out = (out + 1) % BUFFER_SIZE;

        /* consume the item in next consumed */

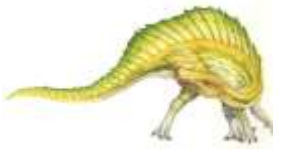
}
```





Interprocess Communication – Shared Memory

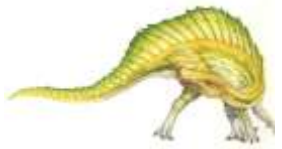
- An area of memory shared among the processes that wish to communicate
- The communication is under the control of the users processes not the operating system.
- Major issues is to provide mechanism that will allow the user processes to synchronize their actions when they access shared memory.
- Synchronization is discussed in great details in Chapter 5.





Interprocess Communication – Message Passing

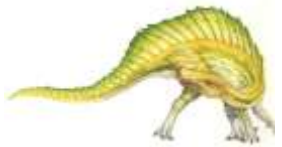
- Mechanism for processes to communicate and to synchronize their actions
- Message system – processes communicate with each other without resorting to shared variables
- IPC facility provides two operations:
  - **`send`**(*message*)
  - **`receive`**(*message*)
- The *message* size is either fixed or variable





Message Passing (Cont.)

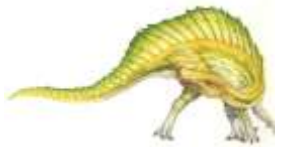
- If processes *P* and *Q* wish to communicate, they need to:
  - Establish a ***communication link*** between them
  - Exchange messages via send/receive
- Implementation issues:
  - How are links established?
  - Can a link be associated with more than two processes?
  - How many links can there be between every pair of communicating processes?
  - What is the capacity of a link?
  - Is the size of a message that the link can accommodate fixed or variable?
  - Is a link unidirectional or bi-directional?





Message Passing (Cont.)

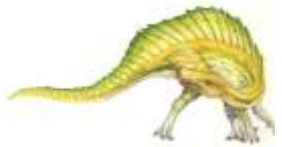
- Implementation of communication link
  - Physical:
    - Shared memory
    - Hardware bus
    - Network
  - Logical:
    - Direct or indirect
    - Synchronous or asynchronous
    - Automatic or explicit buffering





Direct Communication

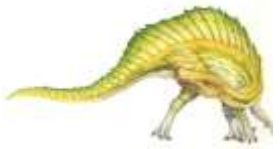
- Processes must name each other explicitly:
  - **`send`** (*P, message*) – send a message to process P
  - **`receive`**(*Q, message*) – receive a message from process Q
- Properties of communication link
  - Links are established automatically
  - A link is associated with exactly one pair of communicating processes
  - Between each pair there exists exactly one link
  - The link may be unidirectional, but is usually bi-directional





Indirect Communication

- Messages are directed and received from mailboxes (also referred to as ports)
  - Each mailbox has a unique id
  - Processes can communicate only if they share a mailbox
- Properties of communication link
  - Link established only if processes share a common mailbox
  - A link may be associated with many processes
  - Each pair of processes may share several communication links
  - Link may be unidirectional or bi-directional



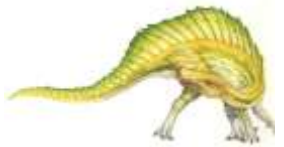


Indirect Communication

- Operations
  - create a new mailbox (port)
  - send and receive messages through mailbox
  - destroy a mailbox
- Primitives are defined as:

  `send`(*A, message*) – send a message to mailbox A

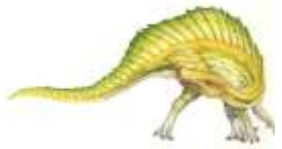
  `receive`(*A, message*) – receive a message from mailbox A





Indirect Communication

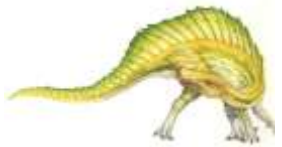
- Mailbox sharing
  - P_1, P_2, and P_3 share mailbox A
  - P_1, sends; P_2 and P_3 receive
  - Who gets the message?
- Solutions
  - Allow a link to be associated with at most two processes
  - Allow only one process at a time to execute a receive operation
  - Allow the system to select arbitrarily the receiver. Sender is notified who the receiver was.





Synchronization

- Message passing may be either blocking or non-blocking
- **Blocking** is considered **synchronous**
  - **Blocking send** -- the sender is blocked until the message is received
  - **Blocking receive** -- the receiver is blocked until a message is available
- **Non-blocking** is considered **asynchronous**
  - **Non-blocking send** -- the sender sends the message and continue
  - **Non-blocking receive** -- the receiver receives:
    - A valid message, or
    - Null message
- Different combinations possible
  - If both send and receive are blocking, we have a **rendezvous**





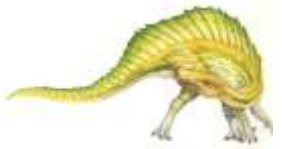
Synchronization (Cont.)

- Producer-consumer becomes trivial

```
message next_produced;
while (true) {
    /* produce an item in next produced */
send(next_produced);
}

message next_consumed;
while (true) {
   receive(next_consumed);

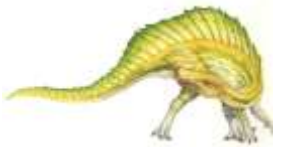
   /* consume the item in next consumed */
}
```





Buffering

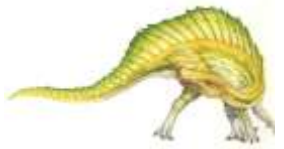
- Queue of messages attached to the link.
- implemented in one of three ways
  1. Zero capacity – no messages are queued on a link.
     Sender must wait for receiver (rendezvous)
  2. Bounded capacity – finite length of *n* messages
     Sender must wait if link full
  3. Unbounded capacity – infinite length
     Sender never waits





Examples of IPC Systems - POSIX

- POSIX Shared Memory
  - Process first creates shared memory segment
    ```
    shm_fd = shm_open(name, O CREAT | O RDWR, 0666);
    ```
  - Also used to open an existing segment to share it
  - Set the size of the object
    ```
    ftruncate(shm fd, 4096);
    ```
  - Now the process could write to the shared memory
    ```
    sprintf(shared memory, "Writing to shared
    memory");
    ```





IPC POSIX Producer

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <fcntl.h>
#include <sys/shm.h>
#include <sys/stat.h>

int main()
{
/* the size (in bytes) of shared memory object */
const int SIZE = 4096;
/* name of the shared memory object */
const char *name = "OS";
/* strings written to shared memory */
const char *message_0 = "Hello";
const char *message_1 = "World!";

/* shared memory file descriptor */
int shm_fd;
/* pointer to shared memory obect */
void *ptr;

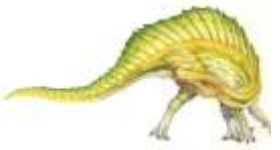
    /* create the shared memory object */
    shm_fd = shm_open(name, O_CREAT | O_RDWR, 0666);

    /* configure the size of the shared memory object */
    ftruncate(shm_fd, SIZE);

    /* memory map the shared memory object */
    ptr = mmap(0, SIZE, PROT_WRITE, MAP_SHARED, shm_fd, 0);

    /* write to the shared memory object */
    sprintf(ptr,"%s",message_0);
    ptr += strlen(message_0);
    sprintf(ptr,"%s",message_1);
    ptr += strlen(message_1);

    return 0;
}
```





IPC POSIX Consumer

```
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <sys/shm.h>
#include <sys/stat.h>

int main()
{
/* the size (in bytes) of shared memory object */
const int SIZE = 4096;
/* name of the shared memory object */
const char *name = "OS";
/* shared memory file descriptor */
int shm_fd;
/* pointer to shared memory obect */
void *ptr;

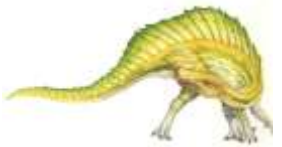
    /* open the shared memory object */
    shm_fd = shm_open(name, O_RDONLY, 0666);

    /* memory map the shared memory object */
    ptr = mmap(0, SIZE, PROT_READ, MAP_SHARED, shm_fd, 0);

    /* read from the shared memory object */
    printf("%s",(char *)ptr);

    /* remove the shared memory object */
    shm_unlink(name);

    return 0;
}
```



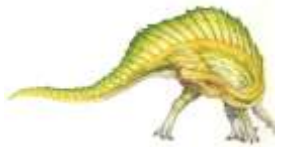


Examples of IPC Systems - Mach

- Mach communication is message based
  - Even system calls are messages
  - Each task gets two mailboxes at creation- Kernel and Notify
  - Only three system calls needed for message transfer

    `msg_send(), msg_receive(), msg_rpc()`
  - Mailboxes needed for commuication, created via

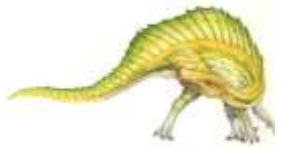
    `port_allocate()`
  - Send and receive are flexible, for example four options if mailbox full:
    - Wait indefinitely
    - Wait at most n milliseconds
    - Return immediately
    - Temporarily cache a message





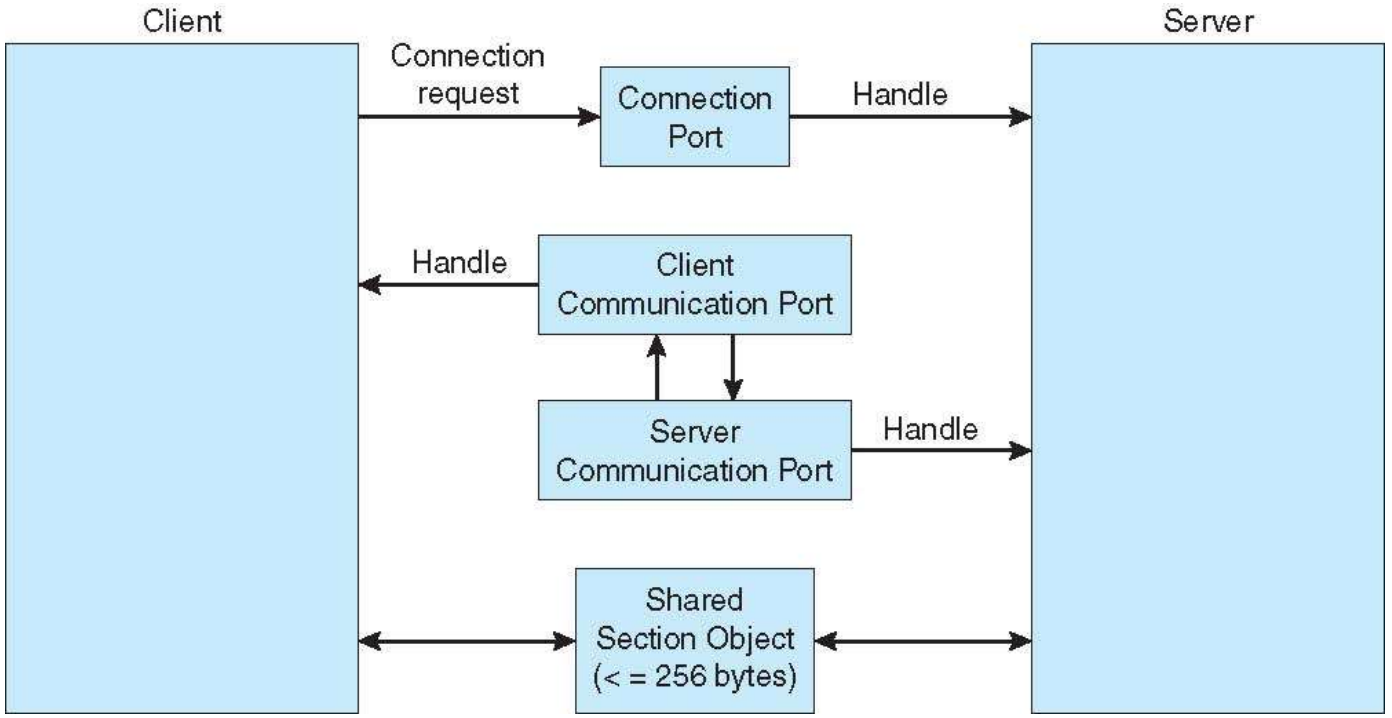
Examples of IPC Systems – Windows

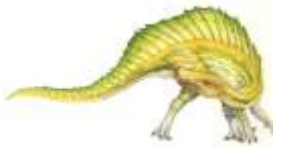
- Message-passing centric via **advanced local procedure call (LPC)** facility
  - Only works between processes on the same system
  - Uses ports (like mailboxes) to establish and maintain communication channels
  - Communication works as follows:
    - The client opens a handle to the subsystem's **connection port** object.
    - The client sends a connection request.
    - The server creates two private **communication ports** and returns the handle to one of them to the client.
    - The client and server use the corresponding port handle to send messages or callbacks and to listen for replies.





Local Procedure Calls in Windows

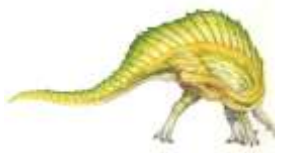






Communications in Client-Server Systems

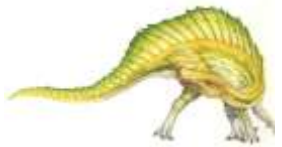
- Sockets
- Remote Procedure Calls
- Pipes
- Remote Method Invocation (Java)





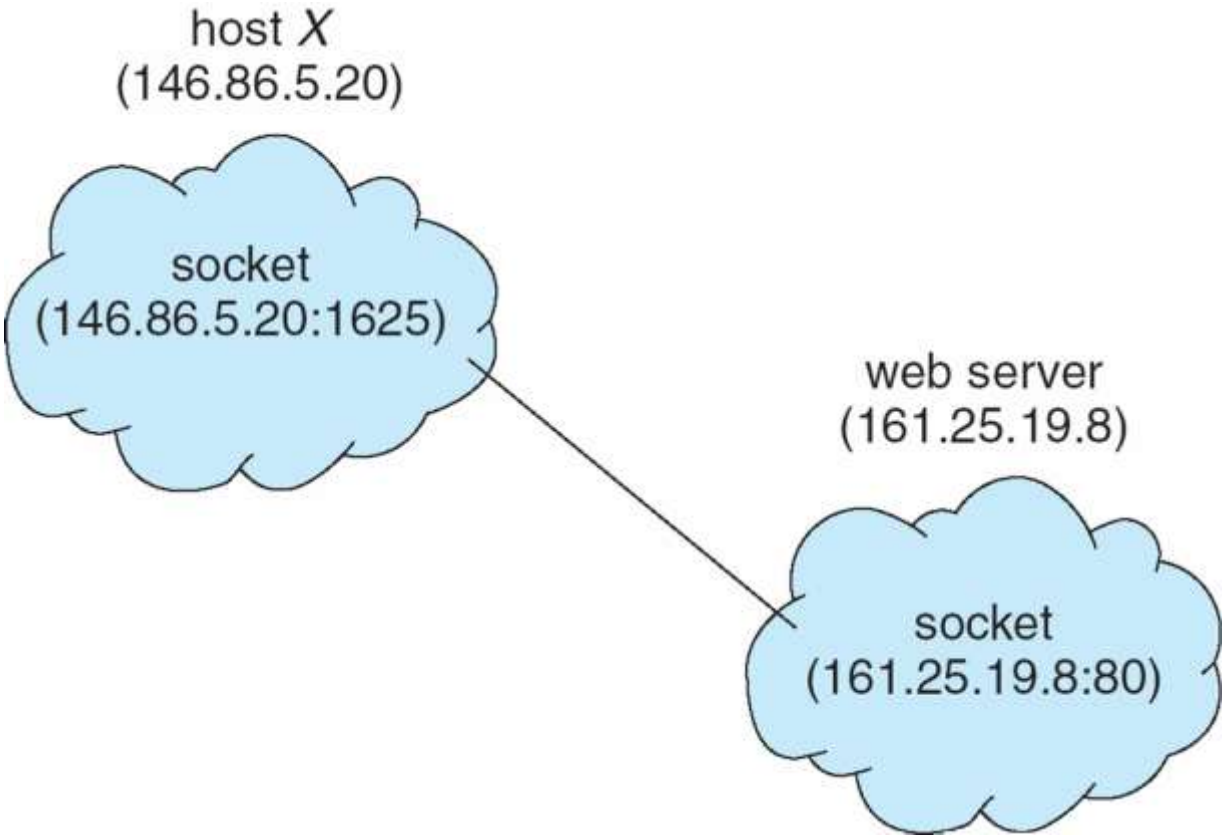
Sockets

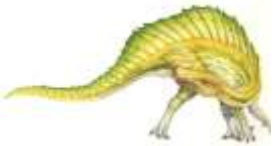
- A **socket** is defined as an endpoint for communication
- Concatenation of IP address and **port** – a number included at start of message packet to differentiate network services on a host
- The socket **161.25.19.8:1625** refers to port **1625** on host **161.25.19.8**
- Communication consists between a pair of sockets
- All ports below 1024 are ***well known***, used for standard services
- Special IP address 127.0.0.1 (**loopback**) to refer to system on which process is running





Socket Communication







Sockets in Java

- Three types of sockets
  - **Connection-oriented** (**TCP**)
  - **Connectionless** (**UDP**)
  - **`MulticastSocket`** class– data can be sent to multiple recipients
- Consider this “Date” server:

```
import java.net.*;
import java.io.*;

public class DateServer
{
   public static void main(String[] args) {
      try {
         ServerSocket sock = new ServerSocket(6013);

         /* now listen for connections */
         while (true) {
            Socket client = sock.accept();

            PrintWriter pout = new
             PrintWriter(client.getOutputStream(), true);

            /* write the Date to the socket */
            pout.println(new java.util.Date().toString());

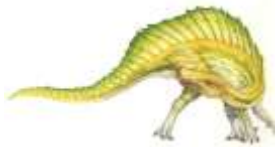
            /* close the socket and resume */
            /* listening for connections */
            client.close();
         }
      }
      catch (IOException ioe) {
         System.err.println(ioe);
      }
   }
}
```





Remote Procedure Calls

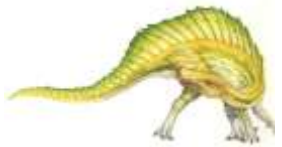
- Remote procedure call (RPC) abstracts procedure calls between processes on networked systems
  - Again uses ports for service differentiation
- **Stubs** – client-side proxy for the actual procedure on the server
- The client-side stub locates the server and **marshalls** the parameters
- The server-side stub receives this message, unpacks the marshalled parameters, and performs the procedure on the server
- On Windows, stub code compile from specification written in **Microsoft Interface Definition Language** (**MIDL**)



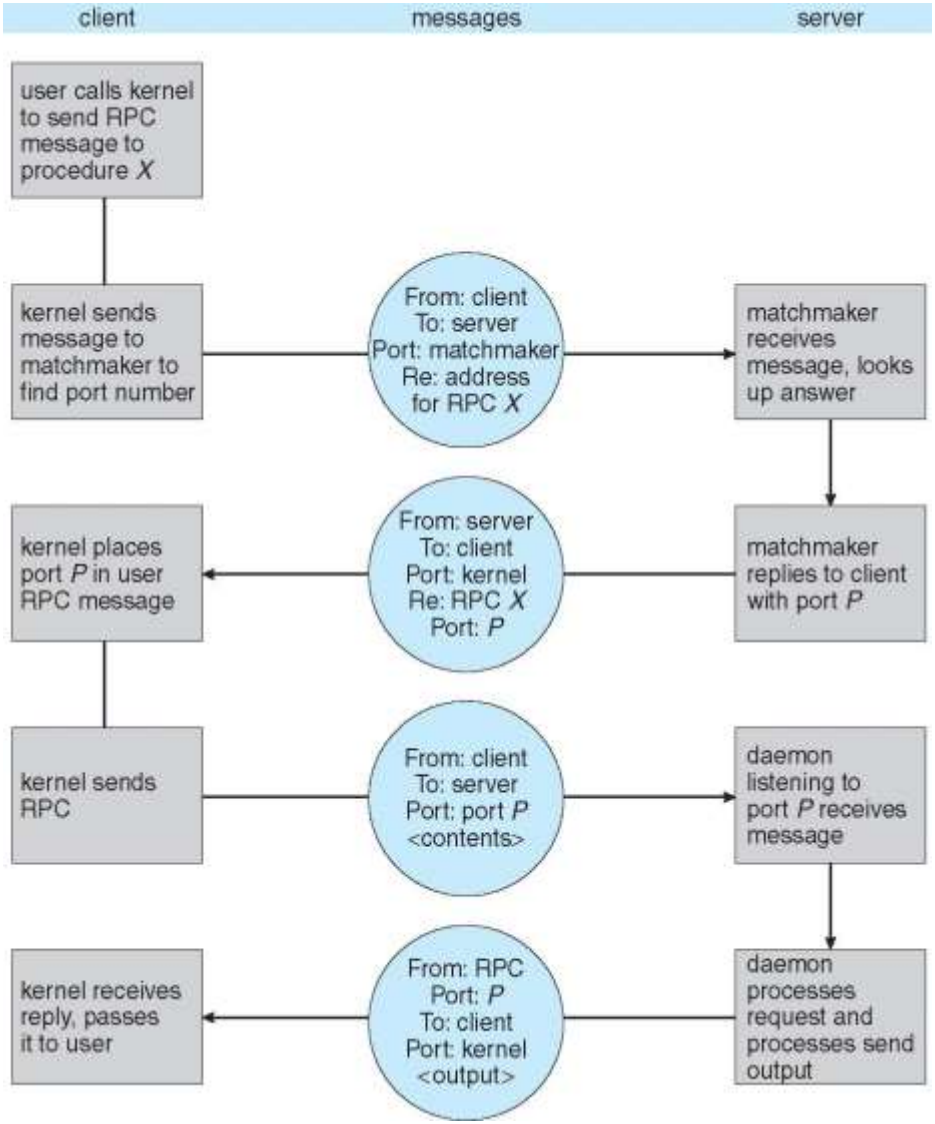


Remote Procedure Calls (Cont.)

- Data representation handled via **External Data Representation** (**XDL**) format to account for different architectures
  - **Big-endian** and **little-endian**
- Remote communication has more failure scenarios than local
  - Messages can be delivered ***exactly once*** rather than ***at most once***
- OS typically provides a rendezvous (or **matchmaker**) service to connect client and server



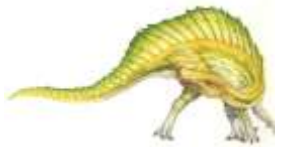
Execution of RPC





Pipes

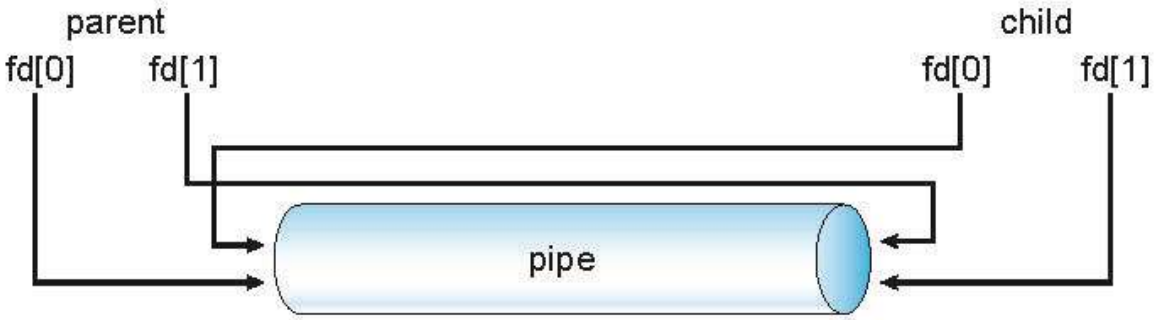
- Acts as a conduit allowing two processes to communicate
- Issues:
  - Is communication unidirectional or bidirectional?
  - In the case of two-way communication, is it half or full-duplex?
  - Must there exist a relationship (i.e., ***parent-child***) between the communicating processes?
  - Can the pipes be used over a network?
- Ordinary pipes – cannot be accessed from outside the process that created it. Typically, a parent process creates a pipe and uses it to communicate with a child process that it created.
- Named pipes – can be accessed without a parent-child relationship.



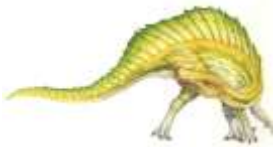


Ordinary Pipes

- Ordinary Pipes allow communication in standard producer-consumer style
- Producer writes to one end (the **write-end** of the pipe)
- Consumer reads from the other end (the **read-end** of the pipe)
- Ordinary pipes are therefore unidirectional
- Require parent-child relationship between communicating processes



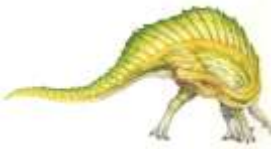
- Windows calls these **anonymous pipes**
- See Unix and Windows code samples in textbook



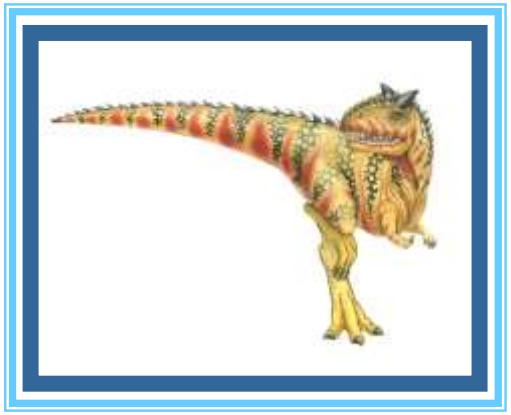


Named Pipes

- Named Pipes are more powerful than ordinary pipes
- Communication is bidirectional
- No parent-child relationship is necessary between the communicating processes
- Several processes can use the named pipe for communication
- Provided on both UNIX and Windows systems



End of Chapter 3



Silberschatz, Galvin and Gagne ©2013

SCHEDULING

SCHEDULING

- When more than one process is ready to run, but only one CPU is available, a choice is to make
- Part of OS that does it is scheduler
- The algorithm it uses is scheduling algorithm

SCHEDULING

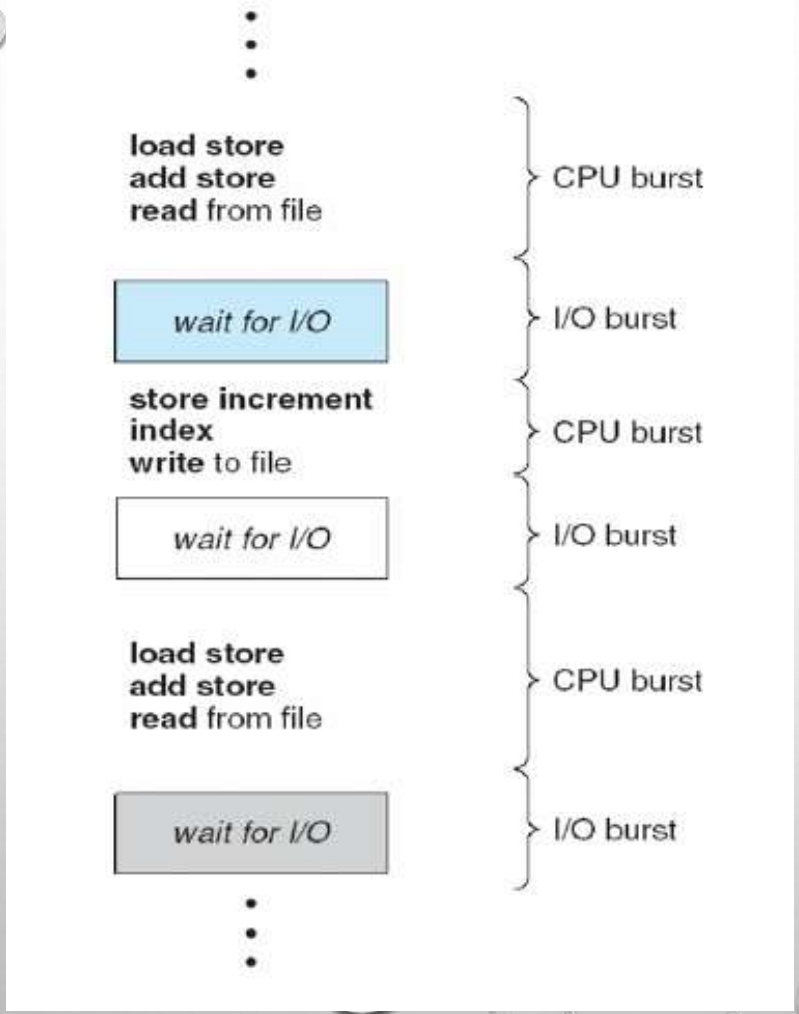
- Efficiency is needed as process switching is costly:
  - Switch from user mode to kernel mode
  - State of current process need to be saved
  - Memory map may be saved
  - A process is selected
  - MMU to be reloaded with memory map of new process
  - New process is started

IMPORTANCE OF SCHEDULING

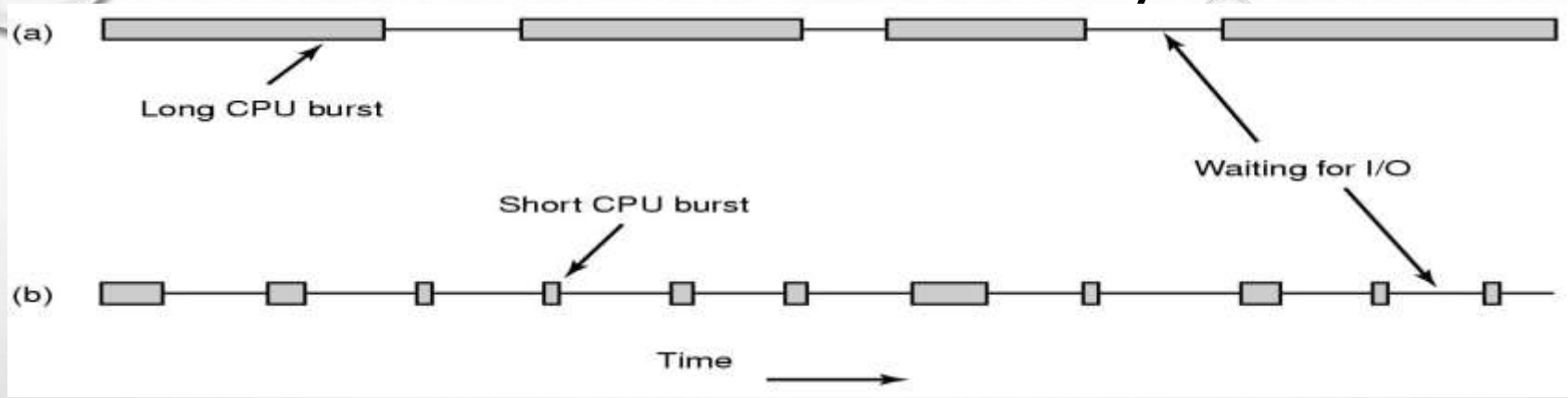
- Good scheduling algorithms can make a big difference
  - Resource utilization
  - Perceived performance & User satisfaction
  - Meeting other system goals (e.g., important tasks being taken care of immediately)

PROCESS BEHAVIOR

- Processes usually alternate bursts of *computing* with *I/O requests.*
- *CPU burst: the amount of time the process uses the processor before it is no longer ready*
- I/O in this sense is when a process enters the blocked state waiting for an external device to complete its work


load store
add store
read from file
CPU burst
wait for I/O
I/O burst
store increment
index
write to file
CPU burst
wait for I/O
I/O burst
load store
add store
read from file
CPU burst
wait for I/O
I/O burst

PROCESS: COMPUTE AND I/O-BOUND



- a CPU-bound process (data encryption/decryption, multimedia encoding)
  - Spend most of the time computing
  - Long CPU bursts => infrequent I/O waits
- an I/O bound process (shell waiting for user commands)
  - Spend most of the time waiting for I/O
  - Short CPU bursts => frequent I/O waits
- Key factor is the **length of CPU burst** not the length of the I/O

PROCESS: COMPUTE AND I/O-BOUND

- As the CPUs get faster, processes tend to get more I/O bound: WHY?
- If a I/O bound process is ready, it should get a chance quickly.
  - Increase resource utilization

WHEN TO SCHEDULE

- When a new process is created:
  - Parent or child? Both are Ready
  - which one to run?
- When a process exits:
  - One of the ready processes should be run
- When a process blocks: Another process has to be selected to run
  - Blocking may occur for:
    - I/O
    - Semaphore

WHEN TO SCHEDULE

- When an I/O interrupt occurs:
  - In case of an interrupt of an I/O device having completed its work, some blocked process may now be ready
- If a h/w clock provides periodic interrupt: A scheduling decision can be made at each (or kth) clock interrupt

PREEMPTIVE & NON-PREEMPTIVE

Classification of Scheduling Algorithm depending on dealing with clock interrupt

- Non-preemptive: Picks a process to run and lets it run until it **blocks** or voluntarily releases the CPU. In effect at each clock interrupt, no scheduling is done.
- Preemptive: Picks a process and lets it run for a maximum of some fixed time. If still running, it is suspended and another is picked.
- Preemptive scheduling requires having a clock

DIFFERENT SYSTEMS, DIFFERENT FOCUSES

All systems

Fairness - giving each process a fair share of the CPU
Policy enforcement - seeing that stated policy is carried out
Balance - keeping all parts of the system busy

Batch systems

Throughput - maximize jobs per hour
Turnaround time - minimize time between submission and termination
CPU utilization - keep the CPU busy all the time

Interactive systems

Response time - respond to requests quickly
Proportionality - meet users' expectations

Real-time systems

Meeting deadlines - avoid losing data
Predictability - avoid quality degradation in multimedia systems

BATCH SYSTEMS

- Users submit their job to the batch system
- Batch system starts user job when appropriate
- User gets notification that job is done
  - No interaction in between
- No users impatiently waiting at terminals for a quick response to a short request
- Used in business world such as Profit calculation at

BATCH SYSTEMS

- Common performance metrics
  - Throughput: number of jobs completed per hour
  - Turnaround time: average time between the submission and completion of a job
- Maximizing Throughput may not necessarily minimize Turnaround time

BATCH SYSTEMS

Algorithms used:

- Non-preemptive
- Preemptive algorithms with long time periods are often acceptable
  - Reduces process switches and improves performance

Representative algorithms:

1. First Come First Serve (FCFS)

FIRST COME FIRST SERVE (FCFS)

- Process that requests the CPU FIRST is allocated the CPU FIRST.
- Also called FIFO
- non-preemptive
- Used in Batch Systems
- Real life analogy?
  - Transaction at Sonali Bank
- Implementation
  - FIFO queues

FCFS EXAMPLE

Process	Duration	Order	Arrival Time
P1	24	1	0
P2	3	2	0
P3	4	3	0

The final schedule:



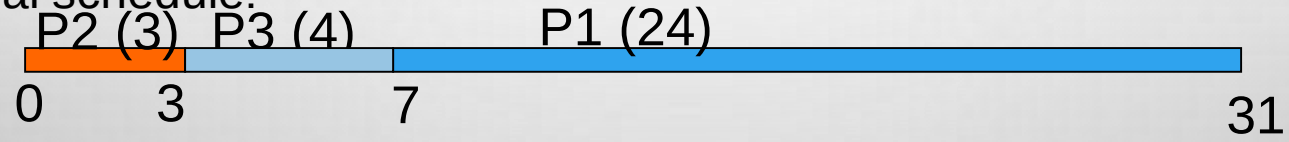
P1 turnaround: 24
P2 turnaround: 27
P3 turnaround: 31

The average turnaround:
(24+27+31)/3 = 27.33

FCFS EXAMPLE 2

Process	Duration	Order	Arrival Time
P1	24	3	0
P2	3	1	0
P3	4	2	0

The final schedule:



P1 turnaround: 31
P2 turnaround: 3
P3 turnaround: 7

The average turnaround:
(31+3+7)/3 = 13.67

ADVANTAGE

- Easy to understand and implement
- Fair for equivalent processes

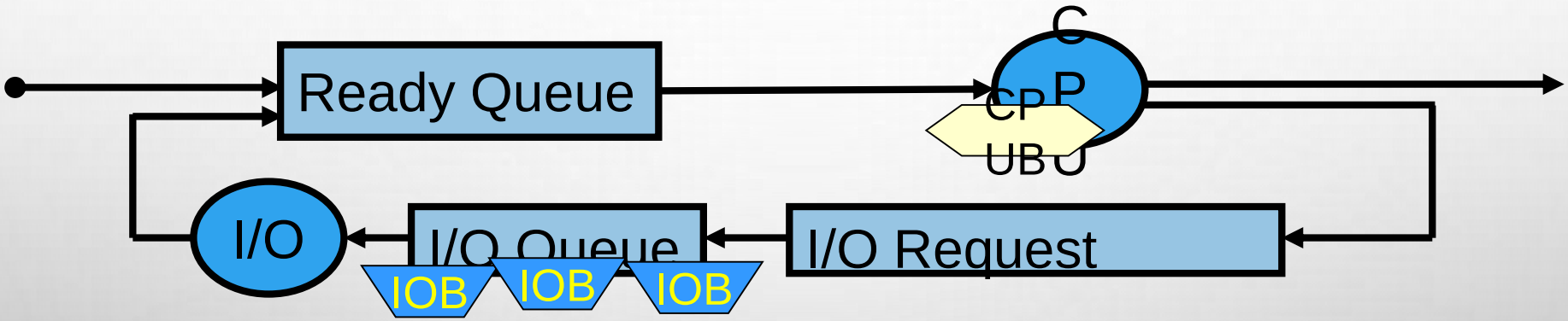
PROBLEMS WITH FCFS

- Non-preemptive
- Non optimal turnaround
- Cannot **utilize** resources in parallel:
  - Assume 1 process CPU bounded and many I/O bounded processes
  - result: **Convoy effect**,
    - low CPU **and** I/O Device utilization
  - Why?

CONVOY EFFECT

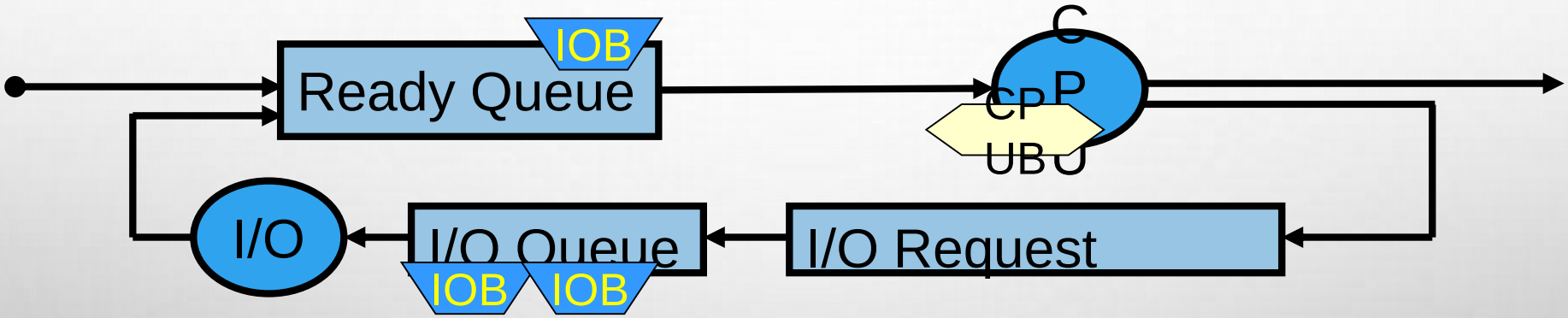
- When the CBP uses the CPU
  - IBPs finish their I/O and move into the ready queue, waiting for the CPU
  - the I/O devices are idle
- When the CBP finally relinquishes the CPU,
  - CBP moves to an I/O device
  - the IBPs pass through the CPU quickly and move back to the I/O queues
  - the CPU is idle

CONVOY EFFECT



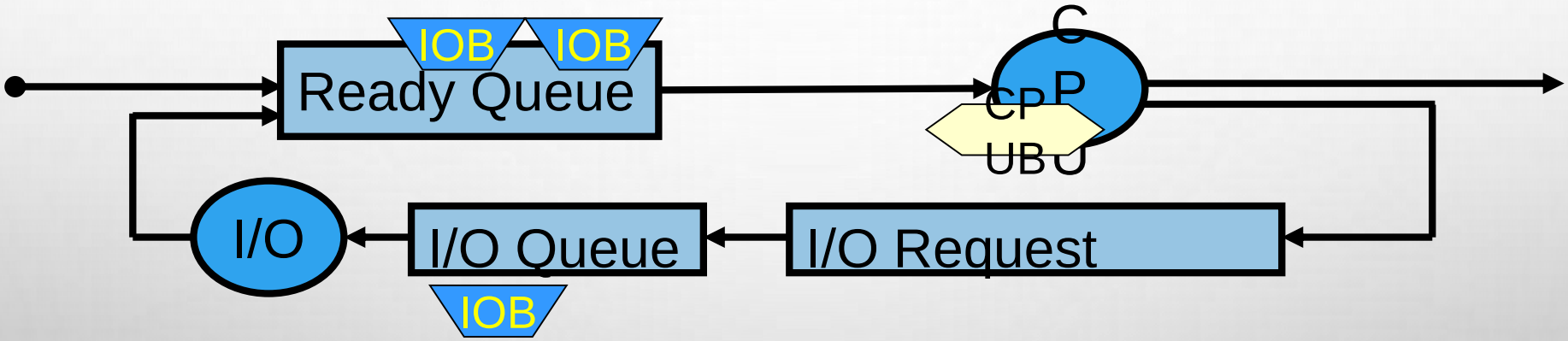
CPU is running CPUB

CONVOY EFFECT



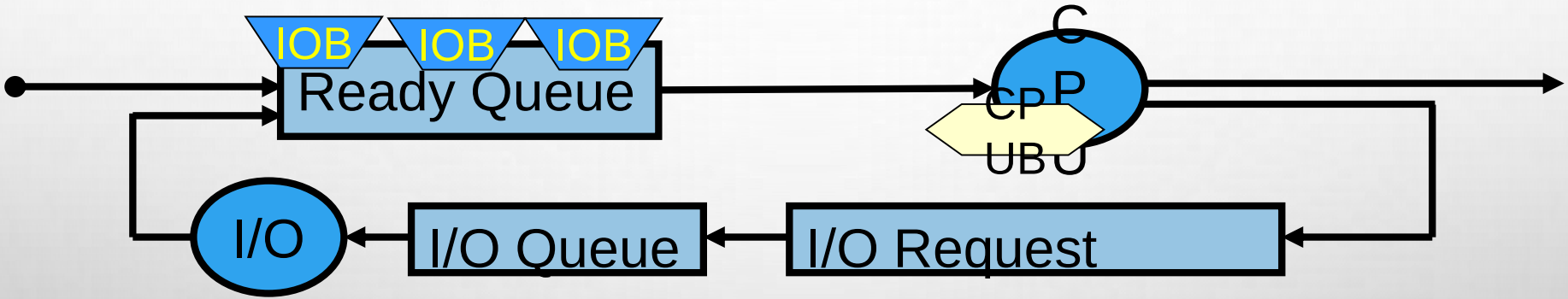
CPU is running CPUB

CONVOY EFFECT



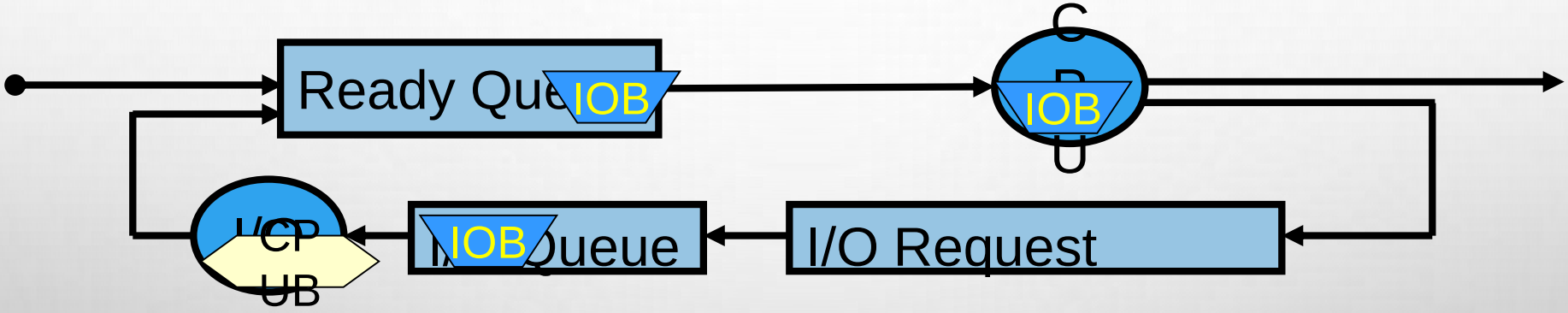
CPU is running CPUB

CONVOY EFFECT



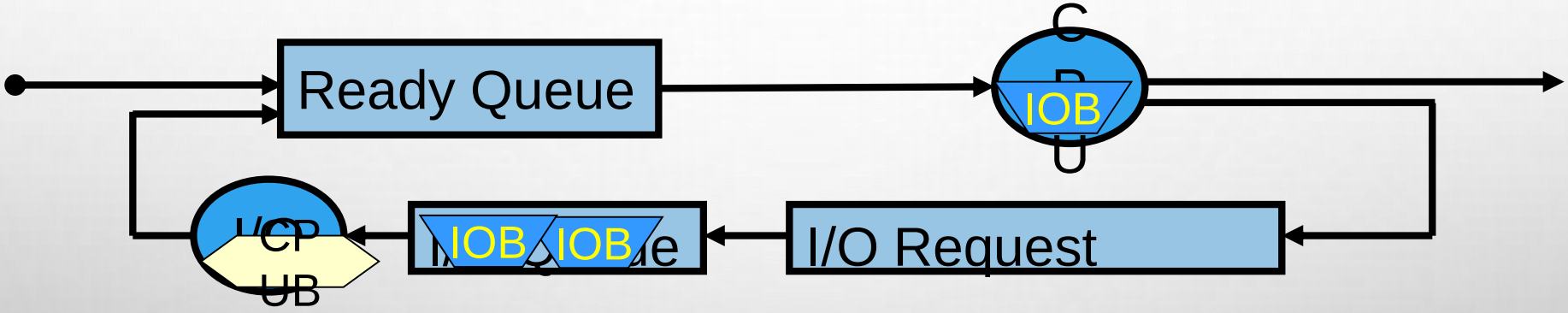
CPU is running CPUB
I/O devices idle

CONVOY EFFECT



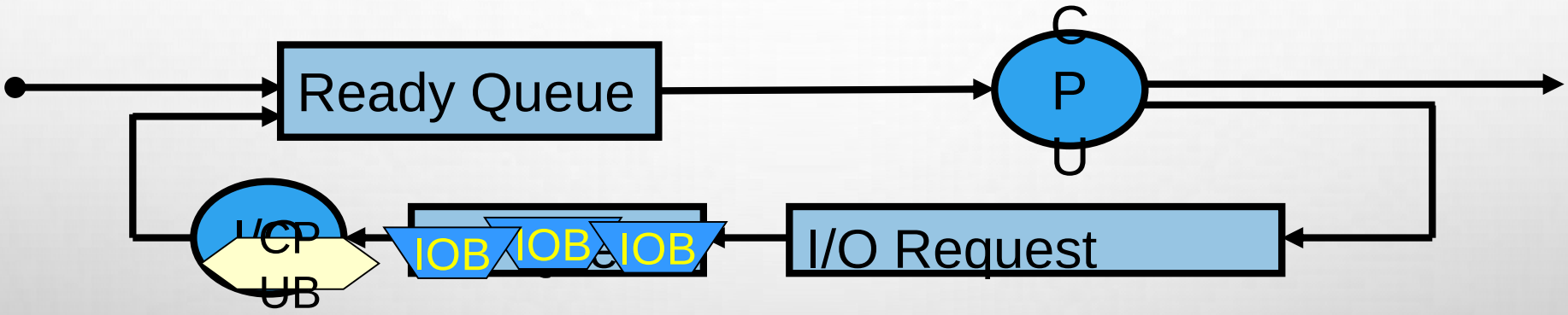
CPUB moves to I/O device

CONVOY EFFECT



I/O Bound jobs take very small amount of CPU time and go for I/O

CONVOY EFFECT



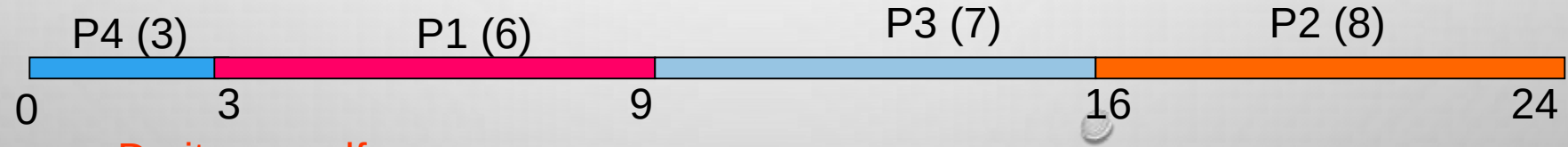
CPU idle

SHORTEST JOB FIRST (SJF)

- Scheduling algorithm in batch systems
- Schedule the job with the shortest run time first
- Requirement: the run time needs to be known in **advance**
- SJF is optimal in terms of turnaround, if **all** jobs arrive at **same time**

SJF: EXAMPLE

Process	Duration	Order	Arrival Time
P1	6	1	0
P2	8	2	0
P3	7	3	0
P4	3	4	0



Do it yourself

P4 turnaround: 3

P1 turnaround: 9

P3 turnaround: 16

Total execution time: 24

The average turnaround:

(3+9+16+24)/4 = 13

COMPARING TO FCFS

Process	Duration	Order	Arrival Time
P1	6	1	0
P2	8	2	0
P3	7	3	0
P4	3	4	0



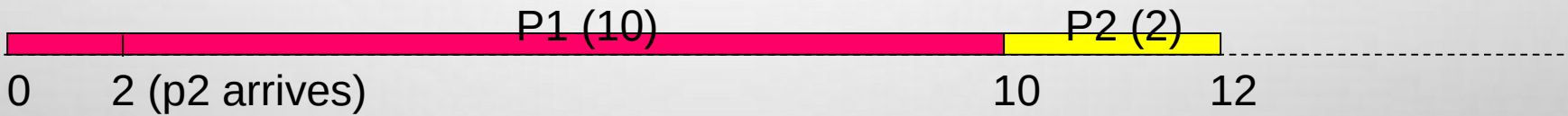
P1 turnaround: 6
P2 turnaround: 14
P3 turnaround: 21
P4 turnaround: 24

The total time is the same.
The average turnaround:
(6+14+21+24)/4 = 16.25
(comparing to 13)

SJF IS NOT ALWAYS OPTIMAL

- SJF OPTIMAL ONLY IF ALL JOBS HAVE ARRIVED AT SCHEDULING TIME

Process	Duration	Order	Arrival Time
P1	10	1	0
P2	2	2	2

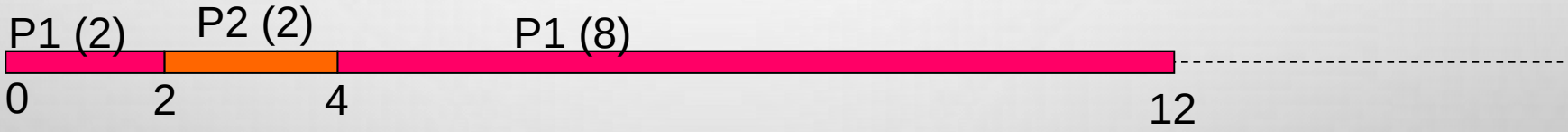


P1 turnaround: 10
P2 turnaround: 10

The average turnaround (AWT):
(10+10)/2 = 10

PREEMPTIVE SJF: SAME EXAMPLE

Process	Duration	Order	Arrival Time
P1	10	1	0
P2	2	2	2



P1 turnaround: 12
P2 turnaround: 2

The average turnaround:
(2+12)/2 = 7

PROBLEM WITH PREEMPTIVE SJF?

- Starvation
  - In some condition, a job is waiting for ever
  - Example: Preemptive SJF
    - Process A with run time of 1 hour arrives at time 0
    - But every 1 minute, a short process with run time of 1 minute arrives
    - Result of Preemptive SJF: A never gets to run

INTERACTIVE SYSTEM

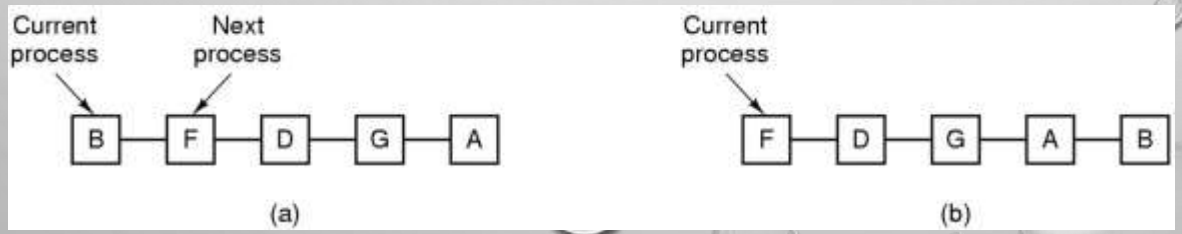
- Example: Servers
  - Serve multiple remote users all of whom are in a big hurry
- Performance Criteria
  - Min response time:
    - amount of time it takes from when a request was submitted until the first response is produced, not output
    - respond to requests quickly

INTERACTIVE SYSTEM

- Algorithms used here usually preemptive
  - Time is **sliced** into quantum (time intervals)
  - Scheduling decision is also made at the beginning of each quantum
- Representative algorithms:
  - Round-robin
  - Priority-based
  - Shortest process time
  - Guaranteed Scheduling
  - Lottery Scheduling

ROUND ROBIN

- **Round Robin (RR)**
  - Often used for timesharing
  - Each process is given a time slice called a ***quantum***
  - It is run for the quantum or until it blocks
  - RR allocates the CPU uniformly (fairly) across participants from ready queue.
- Problem:



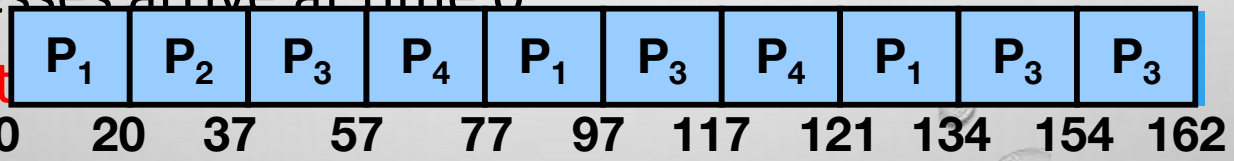
IMPLEMENTING ROUND ROBIN

- Keep the ready queue as a FIFO queue of processes.
- New processes are added to the tail of the ready queue.
- The scheduler
  - picks the first process from the ready queue
  - sets a timer to interrupt after 1 time quantum, and
  - Starts the process.
- When the quantum is over
  - The running process will be put at the **tail** of the ready queue.

RR WITH TIME QUANTUM = 20

Process	Run Time
P_1	53
P_2	17
P_3	68
P_4	24

- All processes arrive at time 0
- The Gant



RR: CHOICE OF TIME QUANTUM

- Performance depends on length of the timeslice
  - Context switching isn't a free operation.
  - If timeslice time is set too high
    - attempting to amortize context switch cost, you get FCFS.
      - i.e. processes will finish or block before their slice is up anyway
      - Poor response time
  - If it's set too low
    - you're spending all of your time context switching between threads.

PRIORITY SCHEDULING

- Each job is assigned a priority
- Select highest priority job to run next
- Rational: higher priority jobs are more important
  - Example: simulation vs. auto save a document
- Problems:
  - Low priority process may starve
- Solution:

ASSIGN PRIORITY

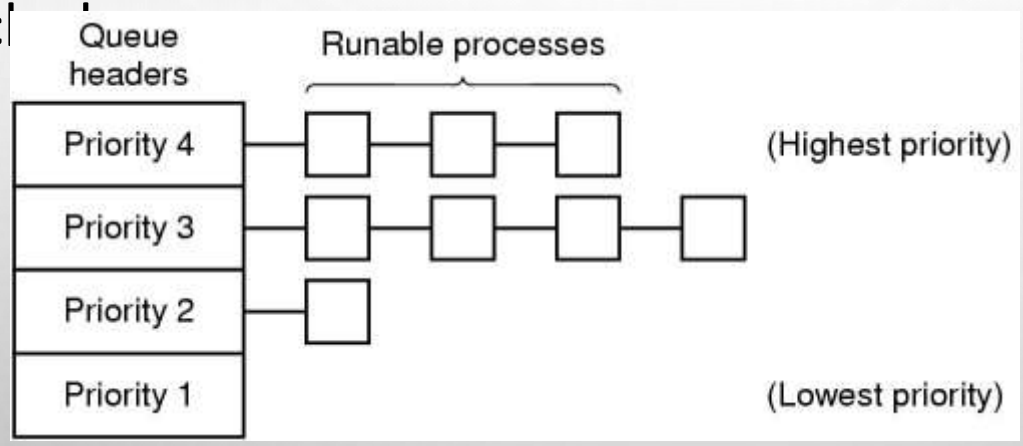
- Two approaches
  - Static (for system with well known and regular application behaviors)
  - Dynamic (otherwise)
- Priority may be based on:
  - Cost to user.
  - Importance of user
  - Percentage of CPU time used in last X hours

EXAMPLE: DYNAMIC PRIORITY ASSIGNMENT

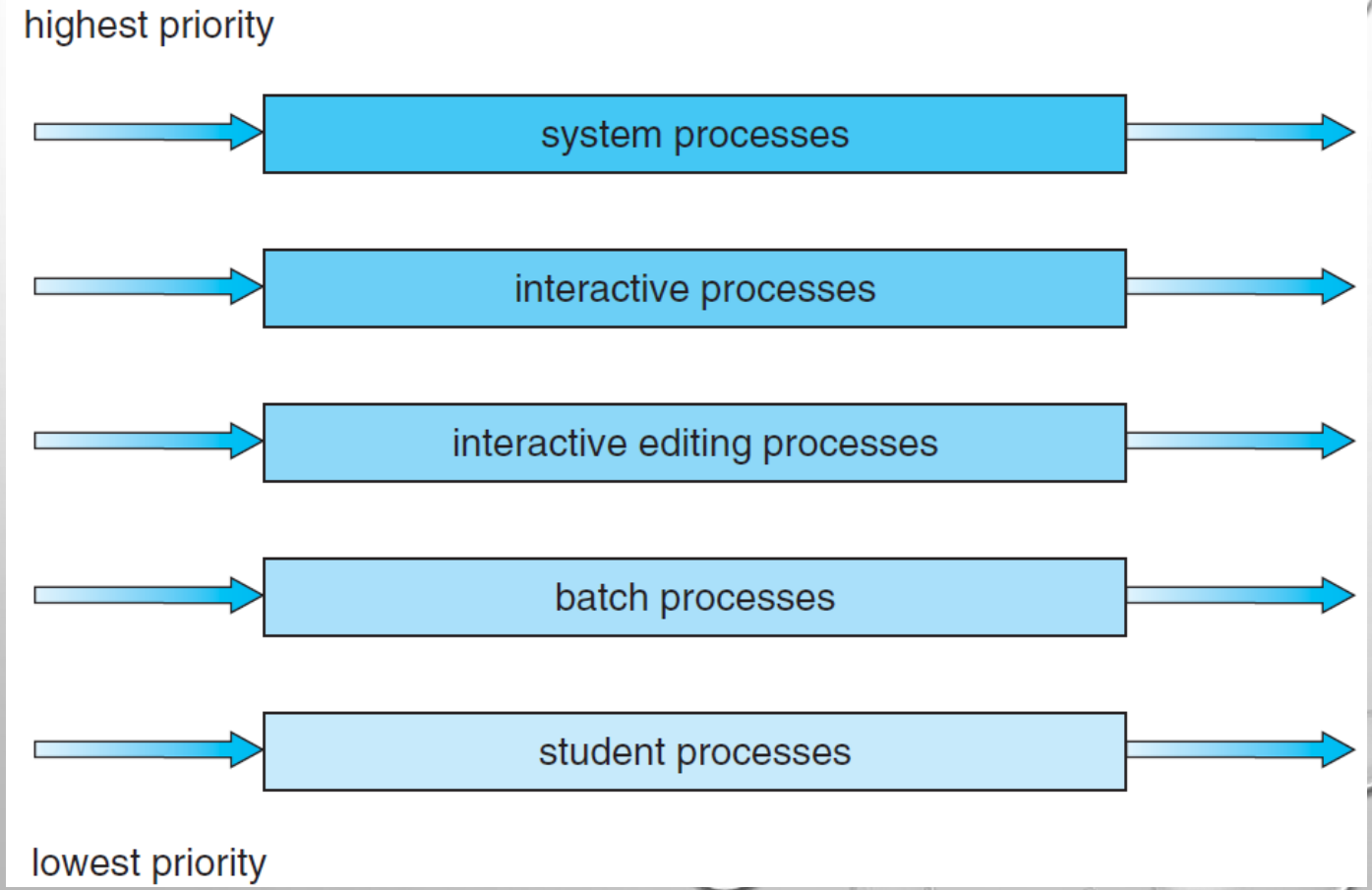
- Whenever highly I/O bound processes wants the CPU it should be given the CPU immediately.
- Why?
- A simple algorithm for giving priority to I/O bound processes is to set the priority to $1/f$
  - f is the fraction of the last quantum used by a process
  - A process that used only 1 msec of its 50 msec quantum would get priority 50
  - A process that used 25 msec of its 50 msec quantum would get priority 2

PRIORITY CLASS

- It is often convenient to group processes into priority classes and use priority scheduling among the classes but RR within eac



PRIORITY CLASS



SHORTEST PROCESS NEXT

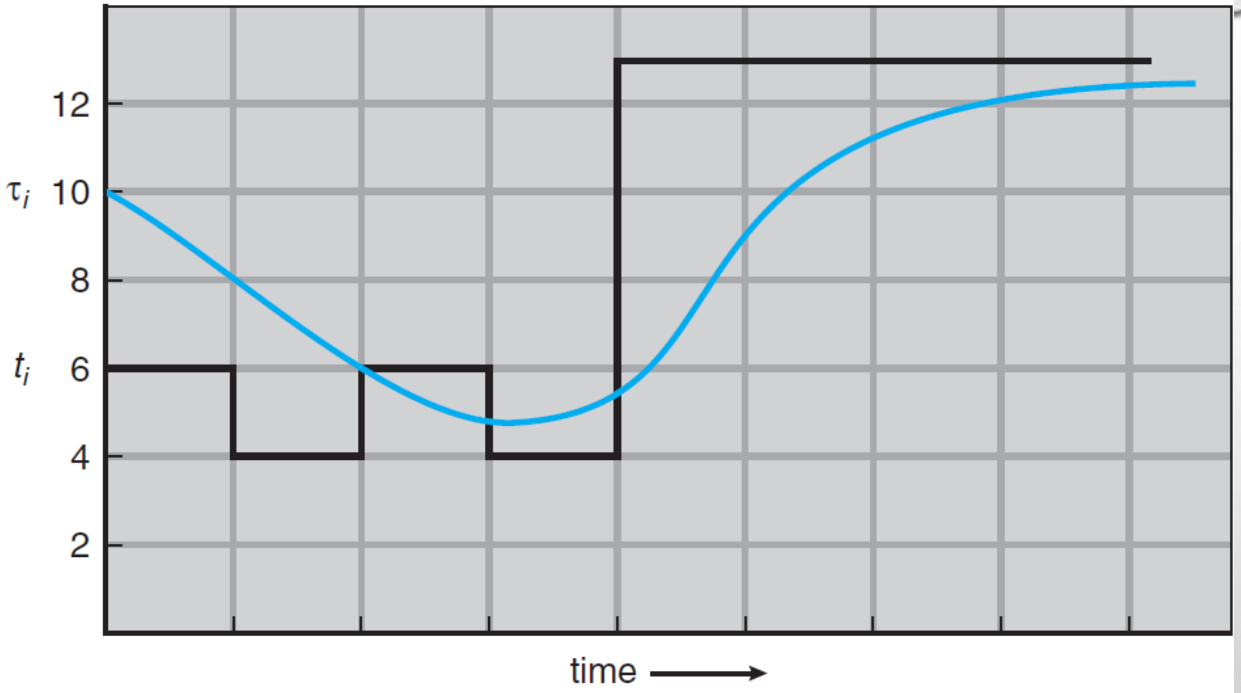
- Let's apply SJF for interactive processes
- General pattern of a interactive process: CPU burst, I/O burst, …
- Let's regard the execution of each CPU burst as a separate "job"
- Now we can minimize overall response time by running the process with shortest "job" first

SHORTEST PROCESS NEXT

- : How to know the length of the next CPU burst?
  - A possible answer: Exponential averaging
  - Make estimate based on past behavior and run the process with the shortest estimated CPU burst
    - Let the current estimated CPU burst is τ_n
    - length of the nth CPU burst t_n
    - predicted value for the next CPU burst $\tau_{n+1} = \alpha t_n + (1 - \alpha)\tau_n$

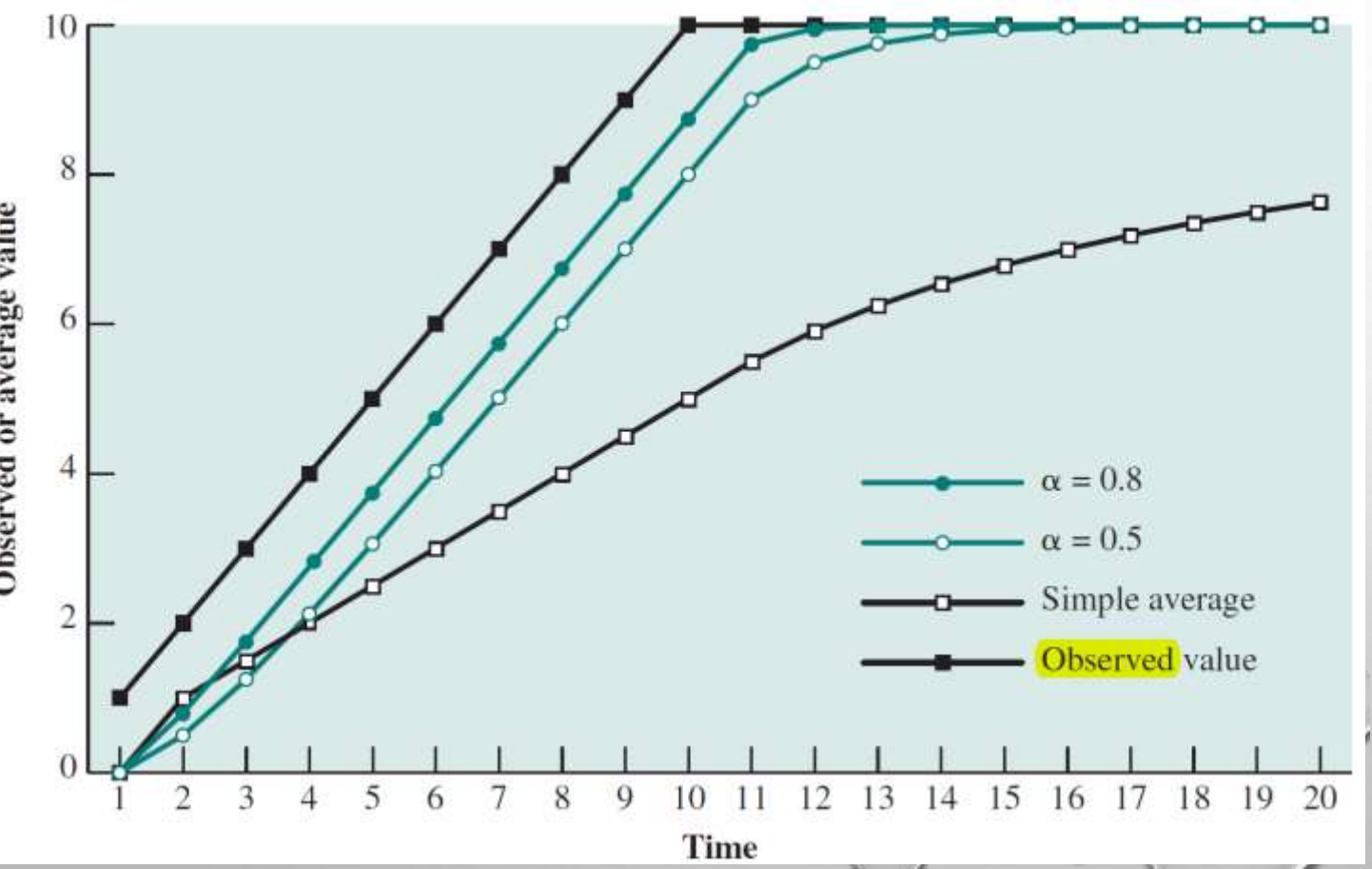
$$\tau_{n+1} = \alpha t_n + (1 - \alpha)\alpha t_{n-1} + \cdots + (1 - \alpha)^j \alpha t_{n-j} + \cdots + (1 - \alpha)^{n+1} \tau_0.$$

EXPONENTIAL AVERAGING



CPU burst (t_i)		6	4	6	4	13	13	13	...
"guess" (τ_i)	10	8	6	6	5	9	11	12	...

EXPONENTIAL AVERAGING



THANKS FOR YOUR PATIENCE

INTERPROCESS COMMUNICATION

INTERPROCESS COMMUNICATION

- Consider shell pipeline
  - `cat chapter1 chapter2 chapter3 | grep tree`
  - 2 processes
  - Information sharing
  - Order of execution

INTERPROCESS COMMUNICATION

- Processes within a system may be **independent** or **cooperating**
- Cooperating process can affect or be affected by other processes
- Reasons for cooperating processes:
  - Information sharing
  - Computation speedup
  - Modularity
  - Convenience
- Cooperating processes require a mechanism to exchange data and information

IPC ISSUES

1. How one process passes information to another ?
2. How to make sure that two or more processes do not get into each other's way when engaging in critical activities?
3. How to do proper sequencing when dependencies are present?

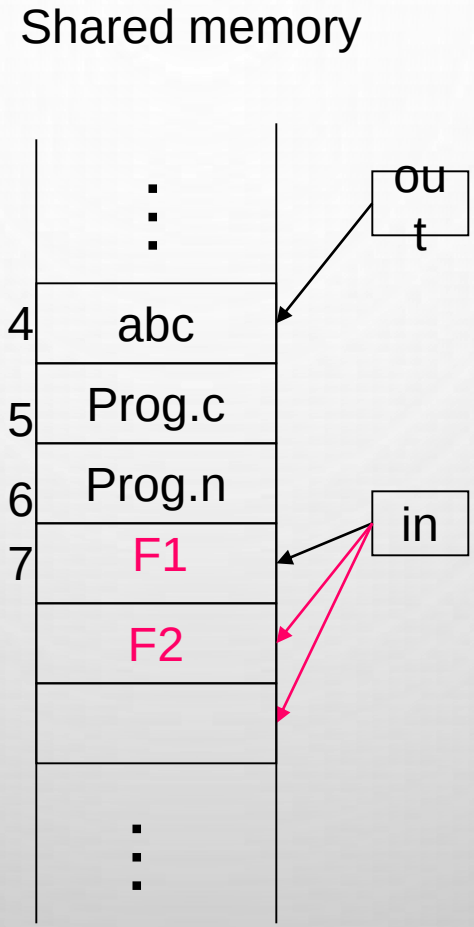
- 1: easy for threads, for processes different approaches (e.g., message passing, shared memory)
- 2 and 3: same problems and same solutions apply for threads and processes
  - Mutual exclusion & Synchronization

SPOOLING EXAMPLE: CORRECT

Process 1

int next_free;

1. next_free = in;
2. Stores F1 into next_free;
3. in=next_free+1



Process 2

int next_free;

4. next_free = in
5. Stores F2 into next_free;
6. in=next_free+1

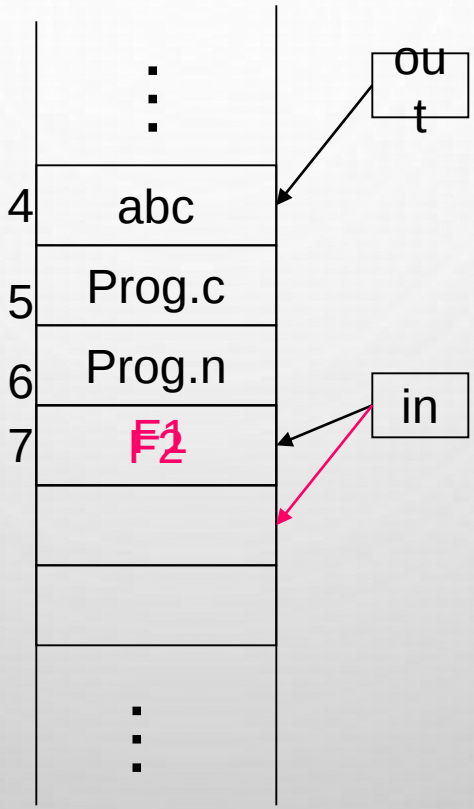
SPOOLING EXAMPLE: RACES

Process 1

int next_free;

1. next_free = in;
3. Stores F1 into next_free;
4. in=next_free+1

Shared memory



Process 2

int next_free;

2. next_free = in /* value: 7 */
5. Stores F2 into next_free;
6. in=next_free+1

BETTER CODING?

- In previous code

```
for(;;){
   int next_free = in;
   slot[next_free] = file;
   in = next_free+1;
}
```

- What if we use one line of code?

```
for(;;){
   slot[in++] = file
}
```

WHEN CAN PROCESS BE SWITCHED?

- After each machine instruction!
- `in++` is a C/C++ statement, translated into three machine instructions:
  - load mem, R
  - inc R
  - store R, mem
- Interrupt (and hence process swichting) can happen in between.

RACE CONDITION

- Two or more processes are reading or writing some shared data and the final result depends on who runs precisely when
- Very hard to Debug

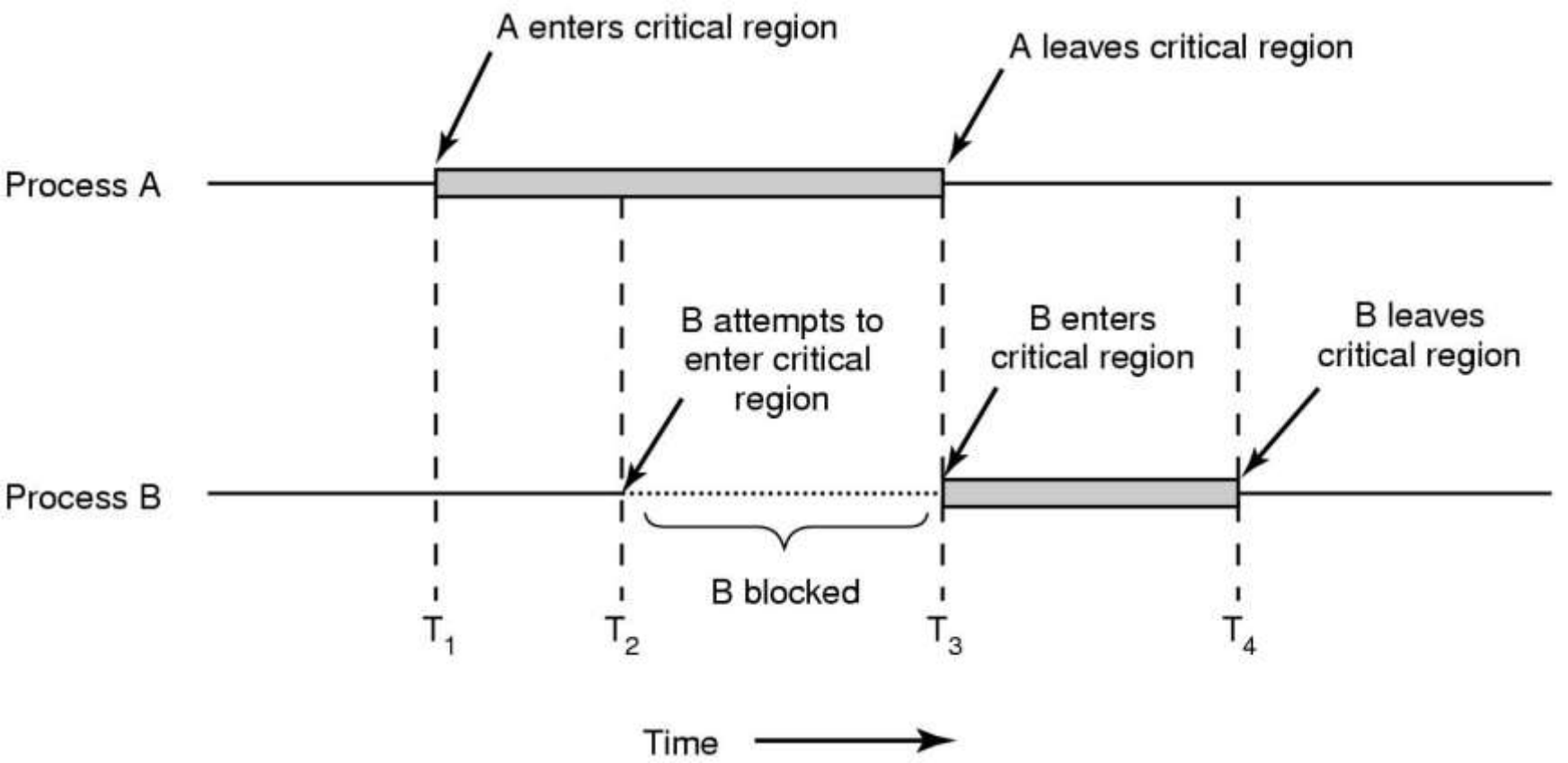
CRITICAL REGION

- That part of the program that do critical things such as accessing shared memory
- Can lead to race condition

SOLUTION REQUIREMENT

1) No two processes simultaneously in critical region
2) No assumptions made about speeds or numbers of CPUs
3) No process running outside its critical region may block another process
4) No process must wait forever to enter its critical region

SOLUTION REQUIREMENT



MUTUAL EXCLUSION WITH BUSY WAITING

- Possible Solutions
  - Disabling Interrupts
  - Lock Variables
  - Strict Alternation
  - Peterson's solution
  - TSL

DISABLING INTERRUPTS

- How does it work?
  - Disable all interrupts just after entering a critical section
  - Re-enable them just before leaving it.
- Why does it work?
  - With interrupts disabled, no clock interrupts can occur
  - No switching can occur
- Problems:
  - What if the process forgets to enable the interrupts?
  - Multiprocessor? (disabling interrupts only affects one CPU)
- Only used inside OS

```
while (true) {
    /* disable interrupts */;
    /* critical section */;
    /* enable interrupts */;
    /* remainder */;
}
```

LOCK VARIABLES

```
int lock = 0;
while (lock);
lock = 1;
//EnterCriticalSection;
   access shared variable;
//LeaveCriticalSection;
lock = 0;
```

Does the above code work?

STRICT ALTERNATION

```
while (TRUE) {
    while (turn != 0)        /* loop */ ;
    critical_region( );
    turn = 1;
    noncritical_region( );
}
```

(a)

```
while (TRUE) {
    while (turn != 1)        /* loop */ ;
    critical_region( );
    turn = 0;
    noncritical_region( );
}
```

(b)

(a) Process 0 **(b) Process 1**

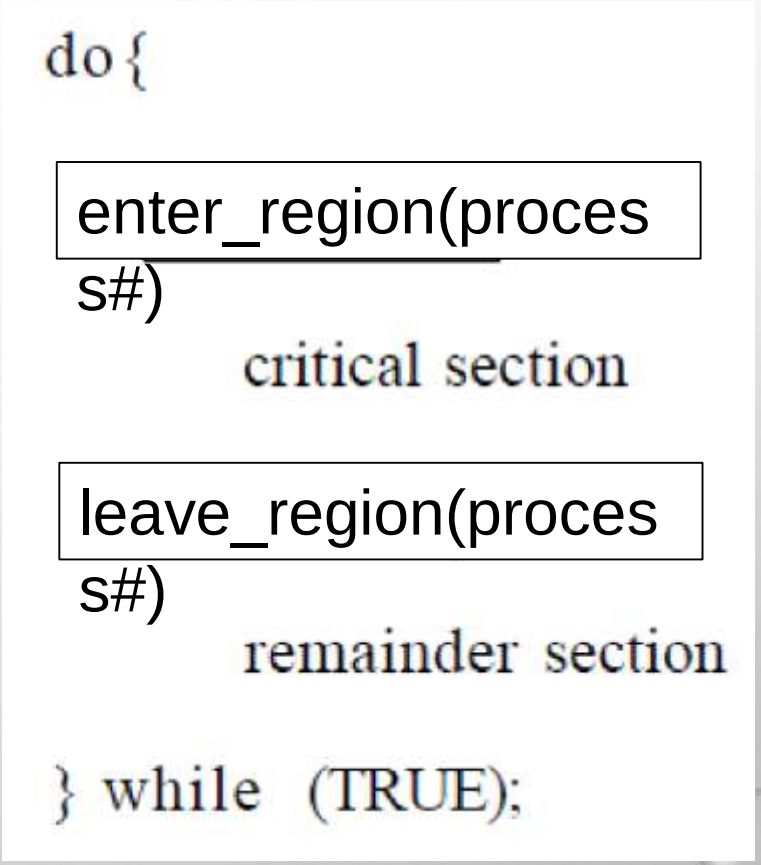
Proposed solution to critical region problem

PROBLEM

- Busy waiting: Continuously testing a variable until some value appear
  - Wastes CPU time
- Violates condition 3
  - When one process is much slower than the other

PETERSON'S SOLUTION

- Consists of 2 procedures
- Each process has to call
  - `enter_region` with its own process # before entering its C.R.
  - And `Leave_region` after leaving C.R.



PETERSON'S SOLUTION (FOR 2 PROCESSES)

```
#define FALSE  0
#define TRUE   1
#define N      2                        /* number of processes */

int turn;                               /* whose turn is it? */
int interested[N];                      /* all values initially 0 (FALSE) */

void enter_region(int process);         /* process is 0 or 1 */
{
     int other;                         /* number of the other process */

     other = 1 – process;               /* the opposite of process */
     interested[process] = TRUE;        /* show that you are interested */
     turn = process;                    /* set flag */
     while (turn == process && interested[other] == TRUE) /* null statement */ ;
}

void leave_region(int process)          /* process: who is leaving */
{
     interested[process] = FALSE;       /* indicate departure from critical region */
}
```

PETERSON'S SOLUTION: ANALYSIS(1)

- Let Process 1 is not interested and Process 0 calls enter_region with 0
- So, turn = 0 and interested[0] = true and Process 0 is in CR
- Now if Process 1 calls enter_region, it will hang there until interested[0] is false. Which only happens when Process 0 calls leave_region i.e. leaves the C.R.

PETERSON'S SOLUTION: ANALYSIS(2)

- Let both processes call enter_region simultaneously
- Say turn = 1. (i.e. Process 1 stores last)
- Process 0 enters critical region: while (turn = = 0 && …) returns false since turn = 1.
- Process 1 loops until process 0 exits: while (turn = = 1 && interested[0] = = true) returns true.
- Done!!

TSL

- Requires hardware support
- TSL instruction: test and set lock
  - Reads content of *lock* into a Register
  - Stores a nonzero value at *lock*.
- CPU executing TSL locks the memory bus prohibiting other CPUs from accessing memory

```
TSL REGISTER,LOCK
```

Indivisible/Atomic

```
enter_region:
        TSL REGISTER,LOCK          | copy lock to register and set lock to 1
        CMP REGISTER,#0            | was lock zero?
        JNE enter_region           | if it was non zero, lock was set, so loop
        RET | return to caller; critical region entered

leave_region:
        MOVE LOCK,#0               | store a 0 in lock
        RET | return to caller
```

BUSY WAITING: PROBLEMS

- Waste CPU time since it sits on a tight loop
- May have unexpected effects:
  - **Priority Inversion Problem**

Example:

- 2 Cooperating Processes: H (high priority) and L (low priority)
- *Scheduling rule:* H is run whenever it is ready
- Let L in C. R. and H is ready and wants to enter C.R.
- Since H is ready it is given the CPU and it starts busy waiting
- L will never gets the chance to leave its C.R.
- H loops forever
- http://research.microsoft.com/en-us/um/people/mbj/Mars_Pathfinder/Mars_Pathfinder.html

SLEEP & WAKEUP

- When a process has to wait, change its state to **BLOCKED/WAITING**
- Switched to **READY** state, when it is OK to retry entering the critical section
- Sleep is a system call that causes the caller to block
  - be suspended until another process wakes it up
- Wakeup system call has one parameter, the process to be awakened.

PRODUCER CONSUMER PROBLEM

- Also called bounded-buffer problem
- Two (*m*+*n*) processes share a common buffer
- One (*m*) of them is (are) **producer**(s): put(s) information in the buffer
- One (*n*) of them is (are) **consumer**(s): take(s) information out of the buffer
- Trouble and solution
  - Producer wants to put but buffer full- Go to **sleep** and **wake up** when consumer takes one or more
  - Consumer wants to take but buffer empty- go to sleep and wake up when producer puts one or more

SLEEP AND WAKEUP

```
#define N 100                                   /* number of slots in the buffer */
int count = 0;                                  /* number of items in the buffer */

void producer(void)
{
     int item;

     while (TRUE) {                             /* repeat forever */
          item = produce_item( );               /* generate next item */
          if (count == N) sleep( );             /* if buffer is full, go to sleep */
          insert_item(item);                    /* put item in buffer */
          count = count + 1;                    /* increment count of items in buffer */
          if (count == 1) wakeup(consumer);     /* was buffer empty? */
     }
}


void consumer(void)
{
     int item;

     while (TRUE) {                             /* repeat forever */
          if (count == 0) sleep( );             /* if buffer is empty, got to sleep */
          item = remove_item( );                /* take item out of buffer */
          count = count - 1;                    /* decrement count of items in buffer */
          if (count == N - 1) wakeup(producer); /* was buffer full? */
          consume_item(item);                   /* print item */
     }
}
```

PRODUCER-CONSUMER PROBLEM

SLEEP AND WAKEUP

```
#define N 100                                   /* number of slots in the buffer */
int count = 0;                                  /* number of items in the buffer */
```

```
void producer(void)
{
     int item;

     while (TRUE) {
          item = produce_item( );
          if (count == N) sleep( );
          insert_item(item);
          count = count + 1;
          if (count == 1) wakeup(consumer);
     }
}
```

```
void consumer(void)
{
     int item;

     while (TRUE) {                                          /
          if (count == 0) sleep( );                          /
          item = remove_item( );                             /
          count = count – 1;                                 /
          if (count == N – 1) wakeup(producer);
          consume_item(item);                                /
     }
}
```


PRODUCER-CONSUMER PROBLEM

SLEEP AND WAKEUP: RACE CONDITION

- **Race condition**
- Unconstrained access to *count*
  - CPU is given to P just after C has read count to be 0 but not yet gone to sleep.
  - P calls wakeup
  - Result is lost wake-up signal
  - Both will sleep forever

SEMAPHORES



- A new variable type
- A kind of generalized lock
  - First defined by Dijkstra in late 60s
  - Main synchronization primitive used in original UNIX
- Semaphores are like integers, except
  - No negative values
  - Only operations allowed are *up* and *down* – can't read or write value, except to set it initially

SEMAPHORES

- Operation “down”:
  - if value > 0; value-- and then continue.
  - if value = 0; process is put to sleep without completing the down for the moment
    - Checking the value, changing it, and possibly going to sleep, is all done as an ***atomic*** *action*.
- Operation “up”:
  - increments the value of the semaphore addressed.
  - If one or more process were sleeping on that semaphore, one of them is chosen by the system (e.g. at random) and is allowed to complete its *down*
    - The operation of incrementing the semaphore and waking up one process is also indivisible
  - No process ever blocks doing an *up.*

SEMAPHORES

- Operations must be atomic
  - Two *down*'s together can't decrement value below zero
  - Similarly, process going to sleep in *down* won't miss wakeup from *up* – even if they both happen at same time

SEMAPHORES

- **Counting semaphore**.
  - The value can range over an unrestricted domain
- **Binary semaphore**
  - The value can range only between 0 and 1.
  - On some systems, binary semaphores are known as mutex locks as they provide mutual exclusion

SEMAPHORES USAGE

1. Mutual exclusion
2. Controlling access to limited resource
3. Synchronization

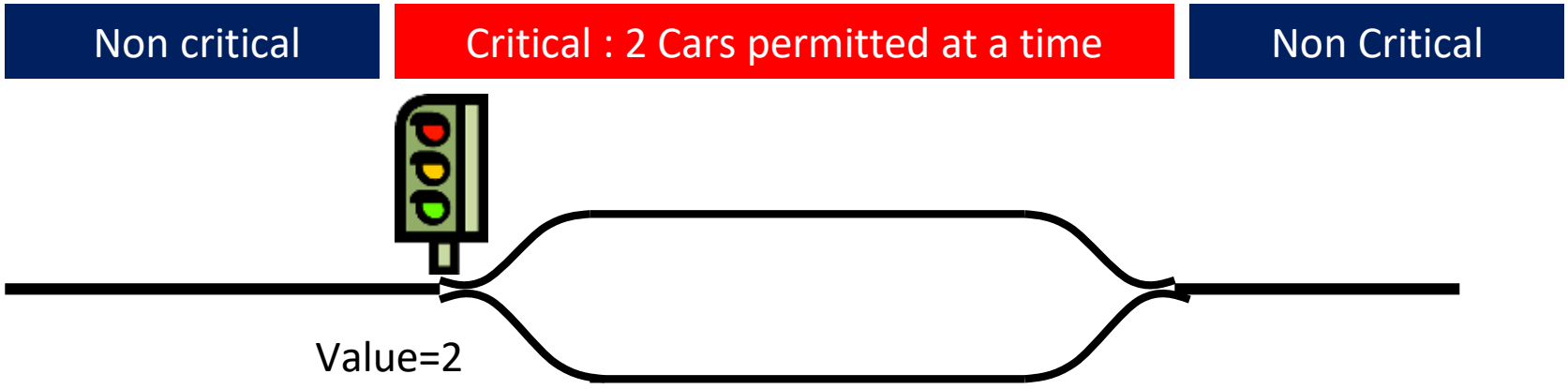
Mutual exclusion

- How to ensure that only one process can enter its C.R.?
- Binary semaphore initialized to 1
- Shared by all collaborating processes
- If each process does a *down* just before entering CR and an *up* just after leaving then mutual exclusion is guarrented

```
do {
        down(mutex);
        // critical section
        up(mutex);
        // remainder section
} while (TRUE);
```

Controlling access to a resource

- What if we want maximum **m** process/thread can use a resource simultaneously ?
- Counting semaphore initialized to the number of available resources
- Semaphore from railway analogy
  - Here is a semaphore **initialized to 2** for resource control:



Synchronization

- How to resolve dependency among processes
- Binary semaphore initialized to 0
- consider 2 concurrently running processes:
  - P1 with a statement S1 and
  - P2 with a statement S2.
  - Suppose we require that S2 be executed only after S1 has completed.

P1

```
S1;
up(synch);
```

P2

```
down(synch);
S2;
```

PRODUCER & CONSUMER

```
#define N 100
typedef int semaphore;
semaphore mutex = 1;
semaphore empty = N;
semaphore full = 0;
```

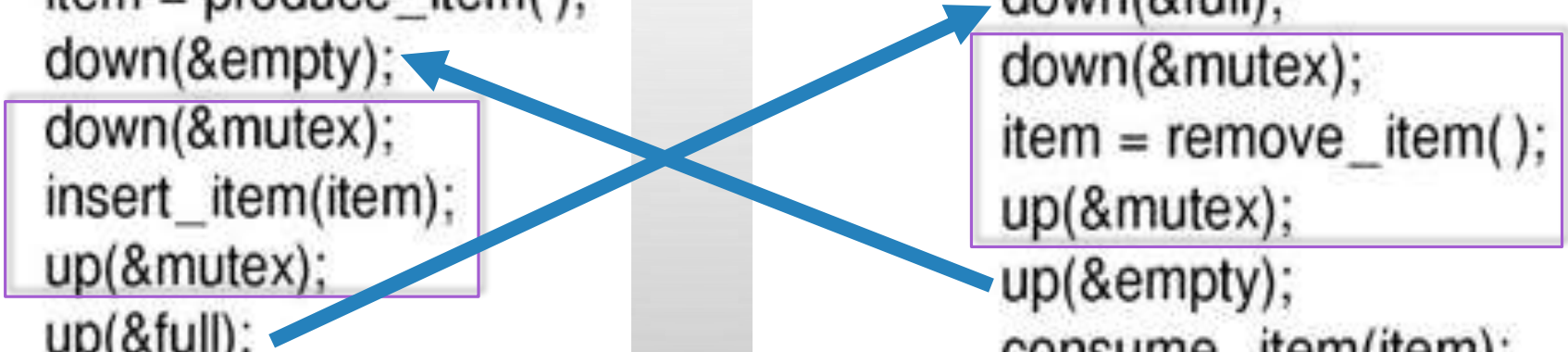
```
/* number of slots in the buffer */
/* semaphores are a special kind of int */
/* controls access to critical region */
/* counts empty buffer slots */
/* counts full buffer slots */
```

```
void producer(void)
{
    int item;

    while (TRUE) {
        item = produce_item( );
        down(&empty);
        down(&mutex);
        insert_item(item);
        up(&mutex);
        up(&full);
    }
}
```

```
void consumer(void)
{
    int item;

    while (TRUE) {
        down(&full);
        down(&mutex);
        item = remove_item( );
        up(&mutex);
        up(&empty);
        consume_item(item);
    }
}
```



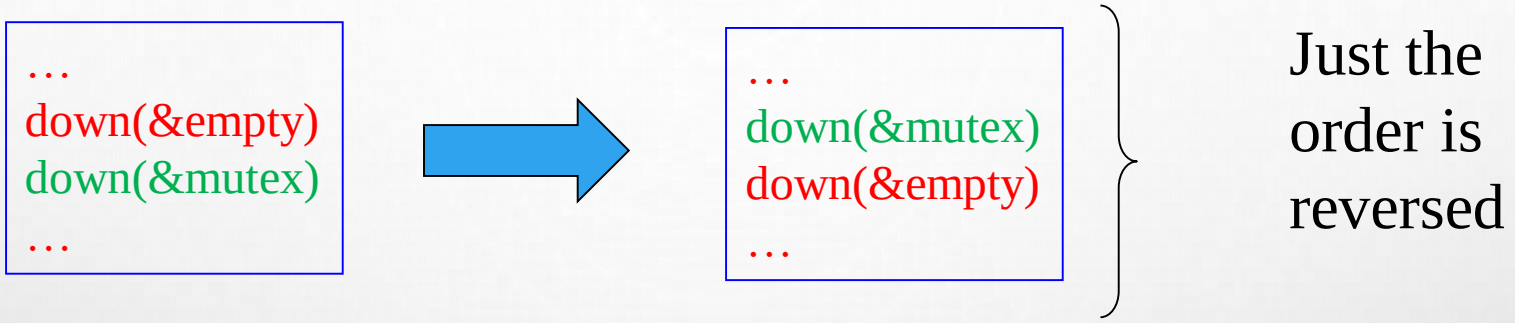
SEMAPHORES IN PRODUCER CONSUMER PROBLEM: ANALYSIS

- 3 semaphores are used
  - *full* (initially 0) for counting occupied slots
  - *Empty* (initially *N*) for counting empty slots
  - *mutex* (initially 1) to make sure that Producer and Consumer do not access the buffer at the same time
- Here 2 uses of semaphores
  - Mutual exclusion (mutex)
  - Synchronization (full and empty)
    - To guarantee that certain event sequences do or do not occur

Block on:	Unblock on:
Producer: insert in full buffer	Consumer: item inserted
Consumer: remove from empty buffer	Producer: item removed

SEMAPHORES: "BE CAREFUL"

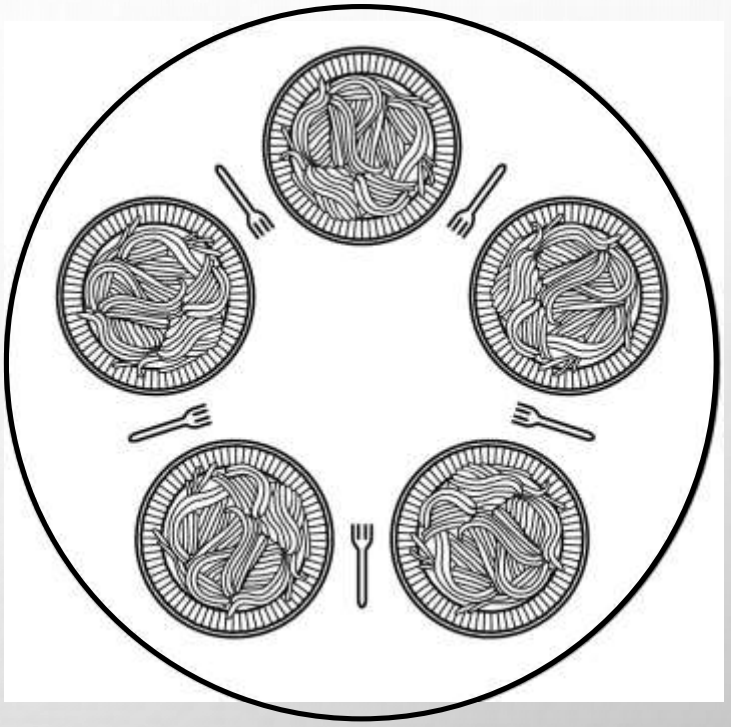
- Suppose the following is done in Producer's code



- If buffer full Producer would block due to down(&empty) with mutex = 0.
- So now if Consumer tries to access the buffer, it would block too due to its down(&mutex).
- Both processes would stay blocked forever: DEADLOCK

DINING PHILOSOPHERS

- Philosophers spend their lives thinking and eating
- Don't interact with their neighbors
- When get hungry try to pick up 2 chopsticks (one at a time in either order) to eat
- Need both to eat, then release both when done
- How to program the scenario avoiding all concurrency problems?



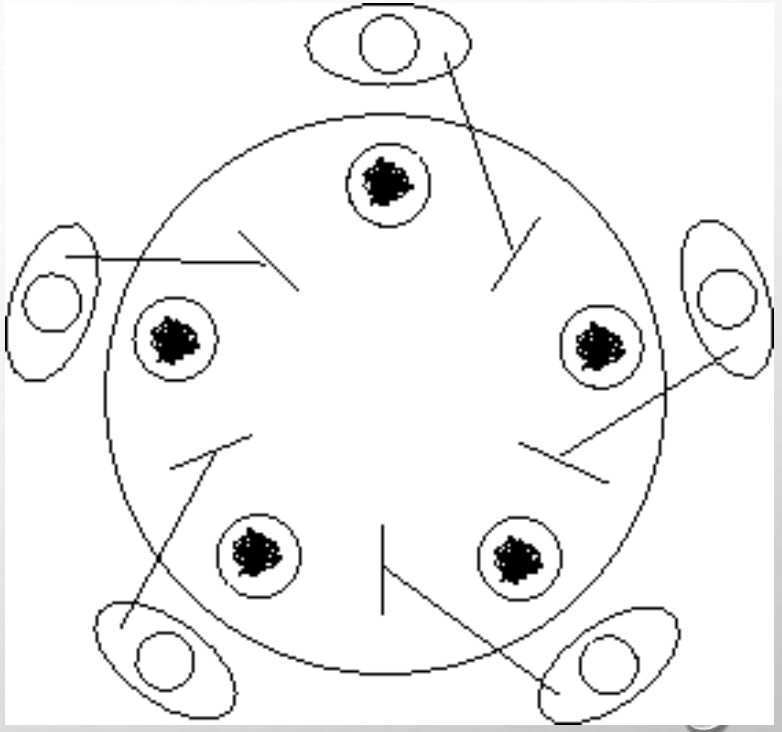
DINING PHILOSOPHERS: A SOLUTION

```
#define N 5                                     /* number of philosophers */

void philosopher(int i)                         /* i: philosopher number, from 0 to 4 */
{
    while (TRUE) {
        think( );                               /* philosopher is thinking */
        take_fork(i);                           /* take left fork */
        take_fork((i+1) % N);                   /* take right fork; % is modulo operator */
        eat( );                                 /* yum-yum, spaghetti */
        put_fork(i);                            /* put left fork back on the table */
        put_fork((i+1) % N);                    /* put right fork back on the table */
    }
}
```

DINING PHILOSOPHERS: PROBLEMS WITH PREVIOUS SOLUTION

- Deadlock may happen
- Does this solution prevents any such thing from happening ?
  - Everyone takes the left fork simultaneously



DINING PHILOSOPHERS: PROBLEMS WITH PREVIOUS SOLUTION

Tentative Solution:

- After taking left fork, check whether right fork is available.
- If not, then return left one, wait for some time and repeat again.

Problem:

- All of them start and do the algorithm synchronously and simultaneously:
- STARVATION (A situation in which all the programs run indefinitely but fail to make any progress)
- Solution: Random wait; but what if the most unlikely of same random number happens?

ANOTHER ATTEMPT, SUCCESSFUL!

```
void philosopher(int i)
{
   while (true)
   {
       think();
       down(&mutex);
       take_fork(i);
       take_fork((i+1)%N);
       eat();
       put_fork(i);
       put_fork((i+1)%N);
       up(&mutex);
   }
}
```

- Theoretically solution is OK- no deadlock, no starvation.
- Practically with a performance bug:
  - Only one philosopher can be eating at any instant: absence of parallelism

FINAL SOLUTION PART 1

```
#define N                5                  /* number of philosophers */
#define LEFT             (i+N–1)%N          /* number of i's left neighbor */
#define RIGHT            (i+1)%N            /* number of i's right neighbor */
#define THINKING         0                  /* philosopher is thinking */
#define HUNGRY           1                  /* philosopher is trying to get forks */
#define EATING           2                  /* philosopher is eating */
typedef int semaphore;                      /* semaphores are a special kind of int */
int state[N];                               /* array to keep track of everyone's state */
semaphore mutex = 1;                        /* mutual exclusion for critical regions */
semaphore s[N];                             /* one semaphore per philosopher */

void philosopher(int i)                     /* i: philosopher number, from 0 to N–1 */
{
    while (TRUE) {                          /* repeat forever */
        think( );                           /* philosopher is thinking */
        take_forks(i);                      /* acquire two forks or block */
        eat( );                             /* yum-yum, spaghetti */
        put_forks(i);                       /* put both forks back on table */
    }
}
```

FINAL SOLUTION PART 2

```
void take_forks(int i)                  /* i: philosopher number, from 0 to N–1 */
{
    down(&mutex);                       /* enter critical region */
    state[i] = HUNGRY;                  /* record fact that philosopher i is hungry */
    test(i);                            /* try to acquire 2 forks */
    up(&mutex);                         /* exit critical region */
    down(&s[i]);                        /* block if forks were not acquired */
}

void put_forks(i)                       /* i: philosopher number, from 0 to N–1 */
{
    down(&mutex);                       /* enter critical region */
    state[i] = THINKING;                /* philosopher has finished eating */
    test(LEFT);                         /* see if left neighbor can now eat */
    test(RIGHT);                        /* see if right neighbor can now eat */
    up(&mutex);                         /* exit critical region */
}

void test(i)                            /* i: philosopher number, from 0 to N–1 */
{
    if (state[i] == HUNGRY && state[LEFT] != EATING && state[RIGHT] != EATING) {
        state[i] = EATING;
        up(&s[i]);
    }
}
```

THE READERS AND WRITERS PROBLEM

- Dining Philosopher Problem: Models processes that are competing for exclusive access to a limited resource
- Readers Writers Problem: Models access to a database
- Example: An airline reservation system- many competing process wishing to read and write-
  - Multiple readers simultaneously- acceptable
  - Multiple writers simultaneously- not acceptable
  - Reading, while write is writing- not acceptable

```
typedef int semaphore;                  /* use your imagination */
semaphore mutex = 1;                    /* controls access to 'rc' */
semaphore db = 1;                       /* controls access to the database */
int rc = 0;                             /* # of processes reading or wanting to */

void reader(void)
{
    while (TRUE) {                      /* repeat forever */
        down(&mutex);                   /* get exclusive access to 'rc' */
        rc = rc + 1;                    /* one reader more now */
        if (rc == 1) down(&db);         /* if this is the first reader ... */
        up(&mutex);                     /* release exclusive access to 'rc' */
        read_data_base( );              /* access the data */
        down(&mutex);                   /* get exclusive access to 'rc' */
        rc = rc − 1;                    /* one reader fewer now */
        if (rc == 0) up(&db);           /* if this is the last reader ... */
        up(&mutex);                     /* release exclusive access to 'rc' */
        use_data_read( );               /* noncritical region */
    }
}


void writer(void)
{
    while (TRUE) {                      /* repeat forever */
        think_up_data( );               /* noncritical region */
        down(&db);                      /* get exclusive access */
        write_data_base( );             /* update the data */
        up(&db);                        /* release exclusive access */
    }
}
```

ISSUE REGARDING THE SOLUTION

- Inherent priority to the readers
- Say a new reader arrives every 2 seconds and each reader takes 5 seconds to do its work. What will happen to a writer?
- Issue regarding second variation
  - Writer don't have to wait for readers that came along after it
  - Less concurrency, lower performance

THANKS FOR YOUR PATIENCE

File Systems

Operating Systems – Study Notes

Topics Covered in these Notes
✓ Files, Attributes & File Directories
✓ File Systems and File Directory Structures
✓ Disk Formatting, Blocks & Addressing
✓ File Allocation Methods (Contiguous, Linked, Indexed)
✓ Magnetic Disk Structure (Platters, Tracks, Sectors)
✓ Disk Access Time (Seek, Rotational Latency, Transfer Time)
✓ Worked Numerical Examples

Compiled and formatted from handwritten class notes

1 Files & File Attributes

Definition: A **file** is a named collection of related information that is recorded on secondary storage.

Each File Has Attributes

*These can be viewed by right-clicking a file and going to **Properties**.*

#	Attribute	Description
1	Name	Identifies the file
2	Extension	Indicates the type of the file (e.g. .docx, .pdf)
3	Size	Amount of storage the file occupies
4	Data	The actual contents of the file
5	Author	Person who created the file
6	Created / Modified / Accessed	Timestamps (dates) for these events
7	Attributes	Read-only, Hidden, etc.
8	Default Program	Program associated to open the file
9	Security Details	Permissions and access control information

File Directory & File System

- **File Directory (Folder):** A collection of files.
- **File System:** The module of the OS which manages, controls, and organizes files and related structures.

*When you open My Computer and click through folders, the file system is what gives you a clean, organized view of the underlying (physically messy) hardware — e.g. an **.exe** file is a perchance/executable file.*

Types of File Systems

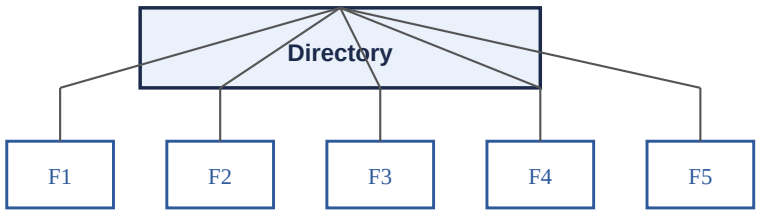
#	File System	Notes
1	FAT32	Old Windows file system; stands for File Allocation Table 32
2	NTFS	New Technology File System — a good, modern feature-rich file system for Windows
3	HFS+	Hierarchical File System — used by Mac OS
4	Ext2 / Ext3 / Ext4	Linux file systems
5	Swap	Used for hardware-managed virtual memory (swap-in / swap-out) — not for regular user files

2 File Directory Structures

When the file system creates a new folder or a new file (e.g. on the Desktop, via right-click → New), the Windows system calls the file system to actually create and run that file. All operations – copy, cut, paste, create new file – happen through the file system.

2.1 Single-Level Directory

The oldest / simplest version — only one folder for the entire disk. All users can see all files.

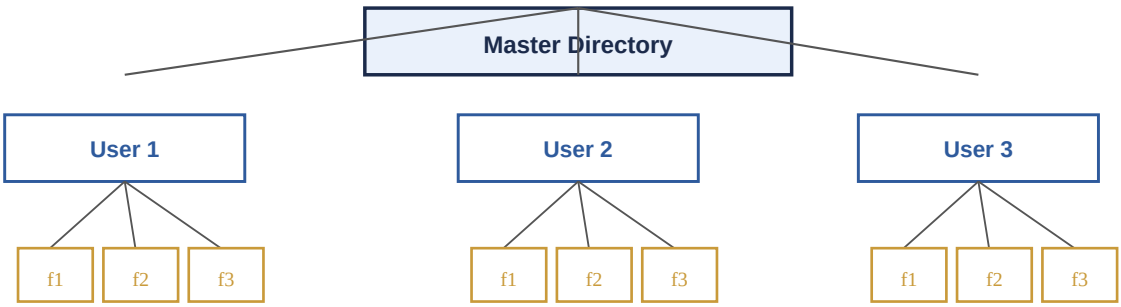


Structure: one Directory pointing directly to all files.

***Problem:** All files need to have unique names in one single directory (no two files can share a name).*

2.2 Two-Level Directory

Each and every user gets their own directory.

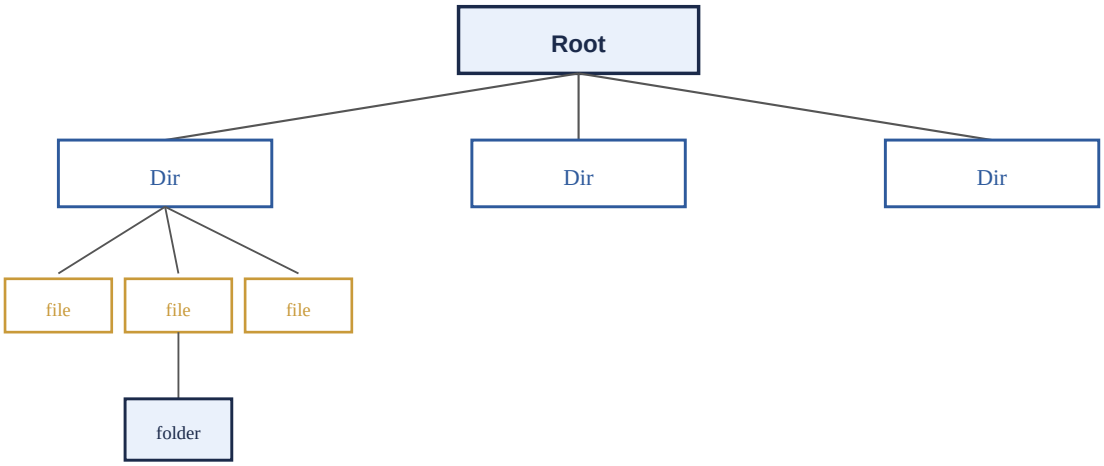


Master directory branches into per-user directories (User 1, User 2, User 3...).

*Files with the **same name** can now exist, as long as they belong to **different (user) directories**.*

2.3 Tree-Structured Directory

What we use now — multiple nested directories under a single **Root Directory**.



Root directory branches into multiple sub-directories, each of which can contain further sub-directories or files — a full multi-level (tree) structure.

- On **Android**, the file system is vulnerable, so the boot directory is kept hidden.
- On Android, a **password/PIN** can be set to change/restrict directory access.

3 Disk Formatting & File Allocation Methods

3.1 Disk Blocks

During **logical formatting**, a disk is divided into multiple **blocks** to store data / files.

If a file is larger than one block, multiple blocks are used. For example, if file size = 4 KB and block size = 1 KB, then 4 blocks are needed to store the file, and the OS needs to know exactly *which* 4 blocks were used.

Worked Example — Maximum File Size

Disk Block Address = 14 bits | Each Disk Block Size = 1 KB

Maximum size of file = Size of disk = 2^{14} × 1 KB = 16 MB

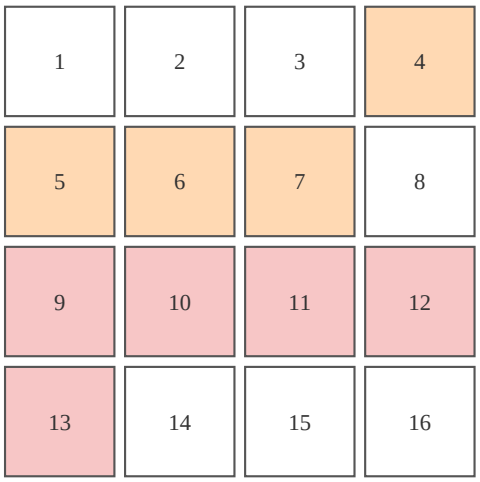
3.2 File Allocation Methods

There are three main ways the OS allocates disk blocks to a file:

- **1. Contiguous Allocation**
- **2. Linked Allocation**
- **3. Indexed Allocation**

3.2.1 Contiguous Allocation

Each file occupies a set of **contiguous (consecutive) blocks** on the disk. A File Allocation Table (FAT) stores the starting block and number of blocks used.



*Disk with 16 blocks. Highlighted: blocks 4–7 (orange) allocated to abc.doc; blocks 9–13 (red) allocated to xyz.pdf — note these blocks are read **serially / sequentially**.*

File Allocation Table (FAT):

File Name	Start Block No.	Number of Blocks
abc.doc	4	4
xyz.pdf	9	5

Performance

Criterion	Contiguous Allocation
Fragmentation	Internal, External
Increase in file size	Not flexible / difficult
Type of access	Sequential, Random / Direct

If a file needs to be edited and its size increases, contiguous allocation may not have free adjacent blocks available — this is a known problem with this method.

3.2.2 Linked Allocation

Each file is a linked list of disk blocks; the blocks may be scattered anywhere on the disk. Each block holds a pointer to the next block, and the last block points to NULL.



Blocks linked with pointers — the final block points to NULL.

File Allocation Table (FAT):

File Name	Start Block	Last Block
abc.doc	4	7
xyz.pdf	2	5

Example: block 4 has a link/pointer to block 12, and so on until the last block.

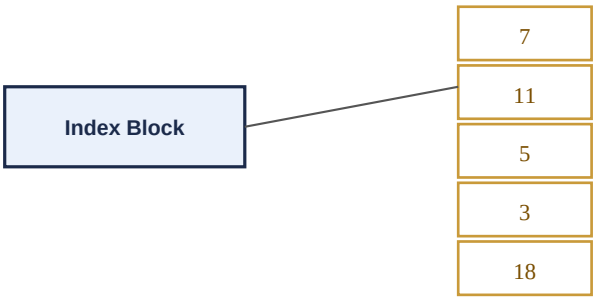
Performance

Criterion	Linked Allocation
Fragmentation	Internal only
Increase in file size	Flexible (easy to extend by linking a new block)
Type of access	Sequential only

Because blocks are scattered and only connected via pointers, jumping directly to the n-th block (random access) is not possible — you must traverse the chain.

3.2.3 Indexed Allocation

All block pointers for a file are collected together in one place: a dedicated **index block** — conceptually like a **book's chapter index**, which lists every page (block) belonging to the file.



The index block stores the list of data-block numbers belonging to the file (e.g. 7, 11, 5, 3, 18).

File Allocation Table (FAT):

File Name	Index Block
xyz.doc	16

For each file, we are needing 1 extra block (the index block itself), in addition to the data blocks.

Performance

Criterion	Indexed Allocation
Fragmentation	Internal
Increase in file size	Flexible
Type of access	Sequential, Random / Direct

Worked Example — Maximum File Size (Indexed Allocation)

Disk Block Address = 16 bits = 2^{16} blocks | Disk Block Size = 1 KB | Index Block = 1 KB

Number of index entries in a single block = 1 KB ÷ 2 B = 512 = 2^9

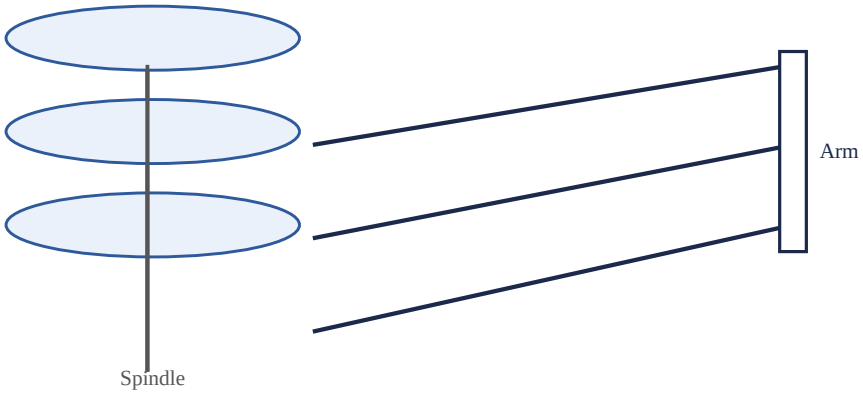
Maximum file size = 2^9 × 1 KB = 512 KB

4 Magnetic Disk Structure

4.1 Disk Types

- **Single CD-like disk** → Optical Disk (e.g. a movie CD). One side is the optical/data side; the opposite side is used for the logo/label.
- **Multiple CD-like disks stacked** → Magnetic Disk.

4.2 Platters & Arm Assembly

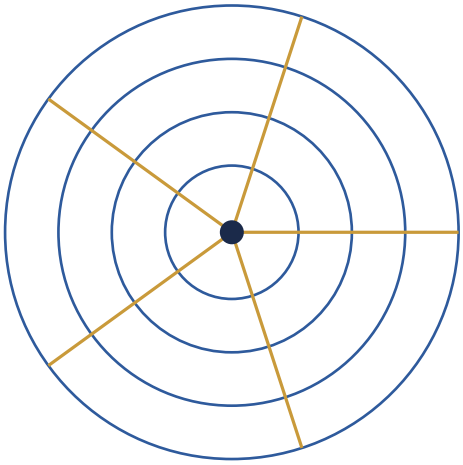


Multiple platters mounted on a spindle; a comb-like arm assembly (with read/write heads) accesses both surfaces of each platter. All arms move together.

A magnetic disk can store information on **both sides (surfaces)** of every platter. For example, 3 platters × 2 surfaces = 6 usable recording surfaces.

An extra CD or a hard disk is often kept as a movie/file copy backup — each disk is like a column-like structure that can be attached alongside others.

4.3 Top View — Tracks & Sectors



*Each platter surface is divided into concentric rings called **tracks**; each track is further divided into **sectors**.*

- Each plate (platter) is divided into **tracks**.
- Each track is divided into **sectors** — e.g. 5 sectors in a particular track.
- Total tracks × sectors/track gives total sectors, e.g. 36 tracks × 5 sectors = total sectors on that surface.

- A **sector** is the smallest addressable unit of the disk — the smallest amount that can be read or written at one moment.

How Read/Write Happens

- Analogous to a gramophone reading black (vinyl) disks.
- All arms of the disk assembly move together as one unit.

5 Disk Access Time

Key Definitions

Term	Meaning
Seek Time	Time required to position the arm over the desired track
Rotational Latency	Time required to rotate the desired sector under the read/write head
Transfer Time	Time required to read or write 1 sector

Disk Access Time Formula

Disk Access Time = Seek Time + Rotational Latency + Transfer Time (1 sector)

Worked Example

Consider a disk with: 16 platters, 2 surfaces per platter, 1K tracks per surface, 2K sectors per track, and 2048 bytes per sector. Disk rotates at 3000 RPM (rotations per minute). Seek Time = 10 ms.

#	Question
1	Find the capacity of the disk
2	Number of bits required for addressing the disk
3	Find the disk access time

1) Capacity of Disk

Capacity = 2 × 16 × 1K × 2K × 2 KB = 128 GB

2) Bits Required for Addressing

Number of sectors = 2 × 16 × 1K × 2K = 2^{26}

Address size = 26 bits

3) Disk Access Time

Rotational Latency: at 3000 rotations per minute (60,000 ms per minute):

Time for 1 rotation = 60,000 ms ÷ 3000 = 20 ms

Rotational Latency = ½ × time for 1 rotation = 20 ÷ 2 = 10 ms

1-Sector Transfer Time = 20 ms ÷ 2K sectors (using rotation time per track)

Disk Access Time = Seek Time + Rotational Latency + 1-Sector Transfer Time
= 10 ms + (20/2) ms + (20/2K) ms
= 20.01 ms

End of Notes